

键值对存储系统

崔立骁

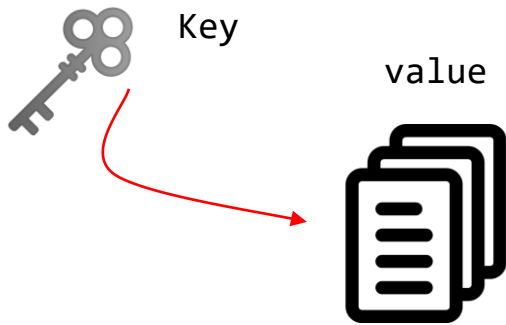
课件部分来源于来源于Animesh Trivedi, Storage Systems (StoSys)
XM_0092, 以及MIT 6.830 Learning Data System Components

What is a Key-Value Store

A simplified data structure to store data and identify with a key (**cache vs store**)

Examples: associate arrays, dictionaries, hash table

Quite popular with web, scalable services



Isn't a file system suppose to store our data?

- FSes create new files, directories for every object
- Web objects are often small, but basic file system inode overheads per directory/files
 - inodes can be a few kBs, if you want to store 64 bytes of data?
- Files/directories are difficult to iterate over quickly
- Range based queries need further auxiliary indexing

Basic Operations

`put(key, value)` : saves a value associated with a key

`value = get (key)` : retrieve the value associated with a key

`delete(key)` : deletes a key (can be equivalent of `put(key, NULL)`)

Batch'ed versions of these commands: `multiget`, `multiput`

Range based queries: `iterate (start_key, end_key);`

Further helper commands: `replace`, `add`, `incr`, `decr`, `merge`, etc.

No single data structure can do all operations efficiently

(see later, the RUM Conjecture)

A Big Design Space

"Key-Value Stores on Flash Storage Devices: A Survey", Krijn Doekemeijer, Animesh Trivedi (2022).

<https://arxiv.org/abs/2205.07975>

Krijn took the course in 2021 :)

arXiv:2205.07975v1 [cs.AR] 11 May 2022

Key-Value Stores on Flash Storage Devices: A Survey

Krijn Doekemeijer
Vrije Universiteit, Amsterdam and Universiteit van Amsterdam

Animesh Trivedi
Vrije Universiteit, Amsterdam

Abstract

Key-value stores (KV) have become one of the main components of the modern storage and data processing system stack. With the increasing need for timely data analysis, performance becomes more and more critical. In the past, these stores were frequently optimised to run on HDD and DRAM devices. However, the last decade saw an increased interest in the use of flash devices because of their attractive properties. Flash is cheaper than DRAM and yet has a lower latency and higher throughput than HDDs. This literature survey aims to highlight the changes proposed in the last decade to optimise key-value stores for flash devices and predict what role these devices might play for key-value stores in the future.

Keywords. Flash storage, SSD, NVMe, Key-value stores, NoSQL, LSM-tree, B-tree, Hash table

1 Introduction

It is estimated that we will generate over 175 zettabytes of data globally by the year 2025 [110]. This is mainly because of the ever-increasing interest in big data, the cloud and the internet of things [71, 110, 130]. As the size of the datasets keeps increasing, so do the demands of the systems that are used to store and process this data. This in turn has caused for an increased interest in optimising the data processing stack. A big part of this stack is used by key-value stores. It is therefore beneficial to look into how key-value stores can be optimised.

Key-value stores are a means of storing data and are radically different from the more traditional RDBs, also known as relational databases [59]. Key-value stores store data as a single collection, where each key is unique and leads to one value. Data can be accessed using these keys with basic operations such as: get, put, delete and scan. Key-value stores can be used for all sorts of applications and are not limited to a particular size or hardware. Some common applications include caching systems [50], messaging applications [21], games [42], web shops [119], SQL backends [48] and time

series management [75].

Traditionally the main storage medium used to store key-value stores was the *Hard Disk Drive* (HDD) [35]. Most data structures and algorithms were thus optimised around the physical properties of these devices. These were among others high latencies, symmetric read and write speeds, slow random access and an infinite number of reads and writes for each block on the HDD. This caused certain HDD specific optimisations such as trying to always write and read sequentially [25, 51, 54, 101, 106].

However, as flash devices became cheaper, many data centres and consumers alike transitioned to flash devices [5, 85]. This made it important to ensure that applications can still be properly used with flash devices and are in addition also optimised for these devices. (Un)fortunately, most of the properties and assumptions that hold for HDDs, do not hold for flash devices. Flash devices have lower latencies, have asymmetric read and write speeds, do not allow for in-place updates, have an all-new erase operation, are indifferent to random reads on the cell level and individual cells have a finite life cycle. The finite life cycle, commonly known as *wear leveling* (WL), can in particular be problematic if unattended. If applications carelessly keep writing to the same cells, the cells will eventually stop working correctly. Lower latencies are also important as lower latencies are becoming more critical for applications [10]. Yet, at the same time lower latencies on flash result in the latency overhead moving to other parts of the key-value store, such as the software that is executed on the host, and therefore require different design considerations [10]. Because of such idiosyncrasies, properly and efficiently using these devices requires a transition [60].

This survey tries to highlight the changes proposed in the last decade for using key-value stores on flash. We will look into various optimisation strategies that can be used to use key-value stores more efficiently on flash. However, first we will take a look at flash and key-value stores themselves. We will then combine the two topics and take a look at the main design concerns that occur when combining them. After having defined the problem space, we will show how these problems

B+ Tree

M-ary tree with sorted (keys-values) stored in leaves

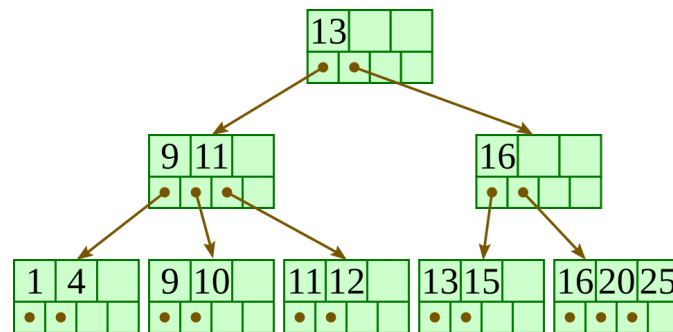
Useful for block-storage devices as it facilitate on-demand node fetching from the storage in a block granularity (e.g., 512 or 4KB)

d-order tree has “d” keys and (d+1) pointers in non-leaf nodes, non-leaf nodes only contains “keys” for pivoting

Self-balancing (by splitting and merging nodes) and distance to all leaves nodes are equal from the root : *every non-leaf, non-root node has at least $\text{floor}(d / 2)$ children, each leaf contains at least $\text{floor}(d / 2)$ keys*

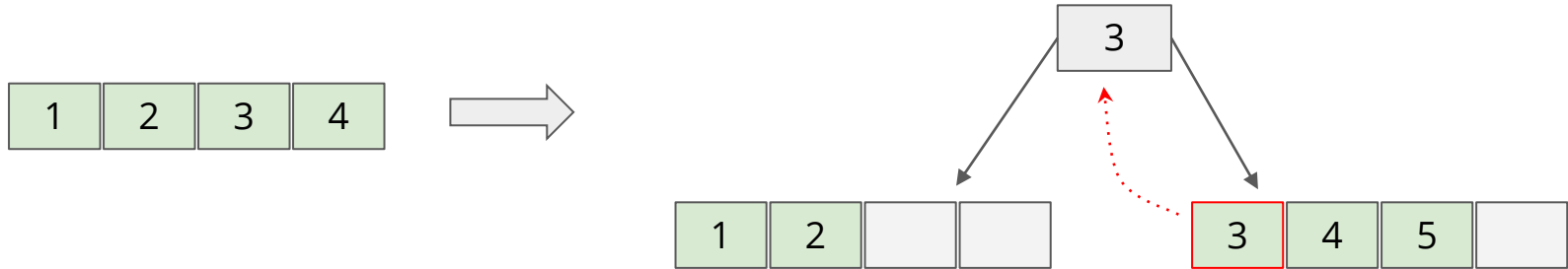
Popular data structure, used in Databases (Oracle, SQL) and file systems (ext4)

Optimized for read-heavy workloads (sorted indexes)



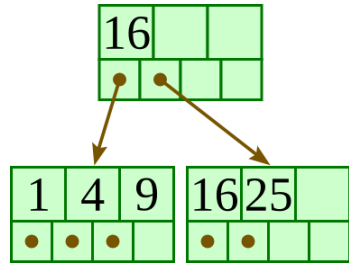
<http://www.cburch.com/cs/340/reading/btree/index.html>

Example: B+ Tree Insertions

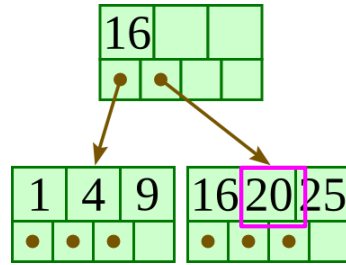


- Split into two, pick the min of left block and push up
- If it was a non-leaf split, then remove the key from low levels

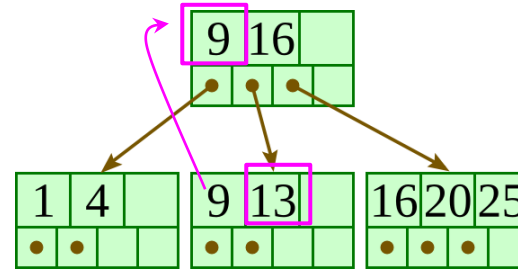
Example: B+ Tree Insertions



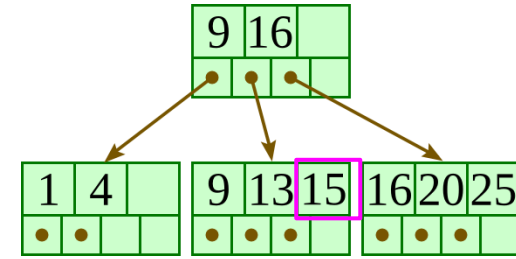
Initial



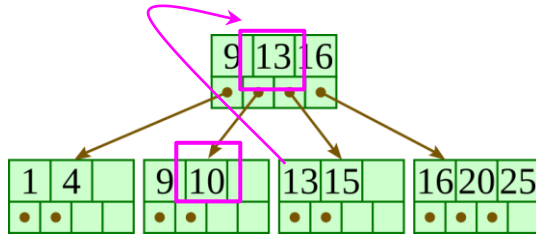
Insert 20



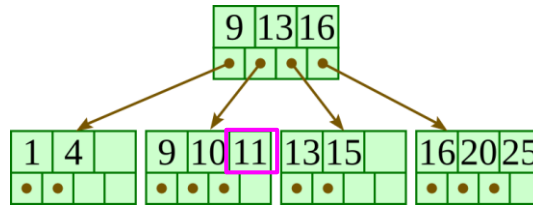
Insert 13



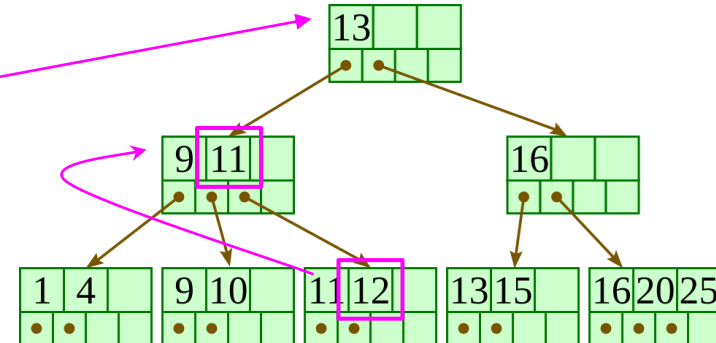
Insert 15



Insert 10

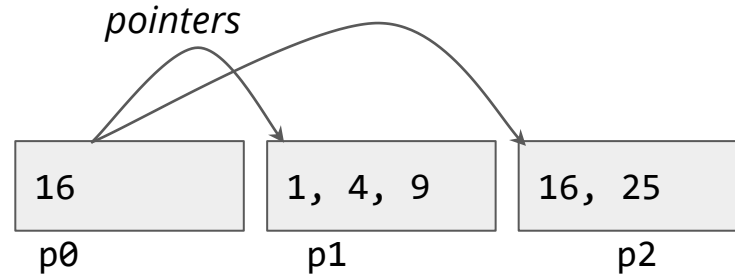
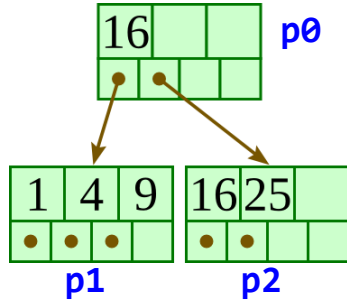


Insert 11



Insert 12

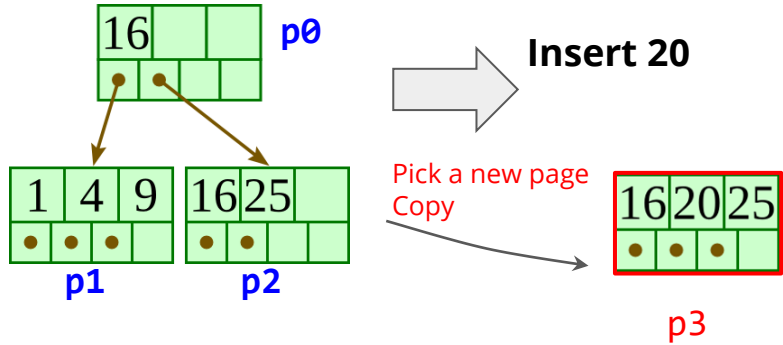
Example: B+ Tree Insertions on NAND Flash



NAND flash pages, the same layout used with HDD too

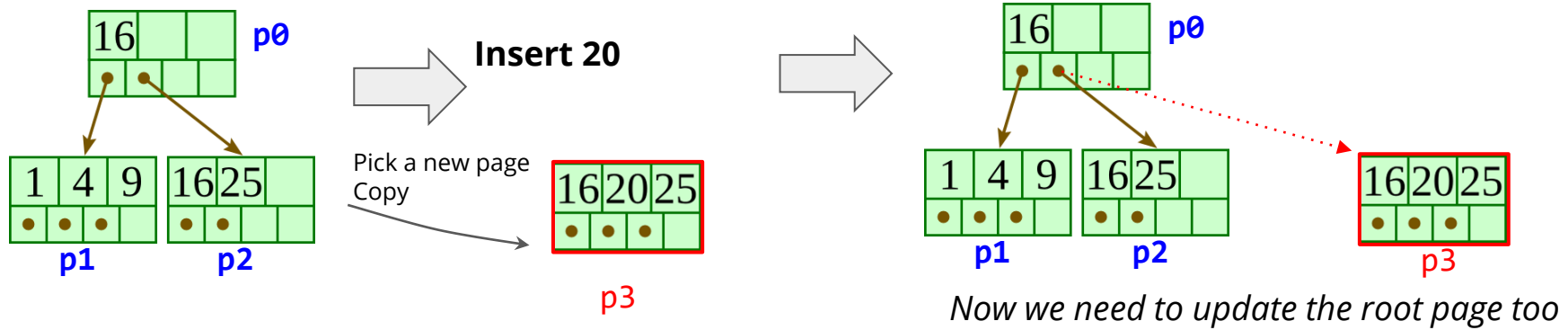
- Whole pages can be read in a single go
- Large sequential transfers, good performance
- All values sorted, so we know which page to load for which node

Example: B+ Tree Insertions on NAND Flash

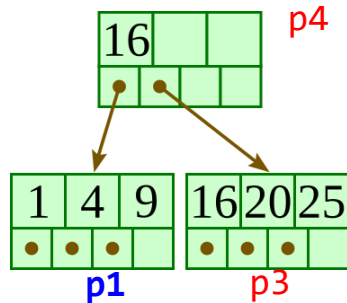
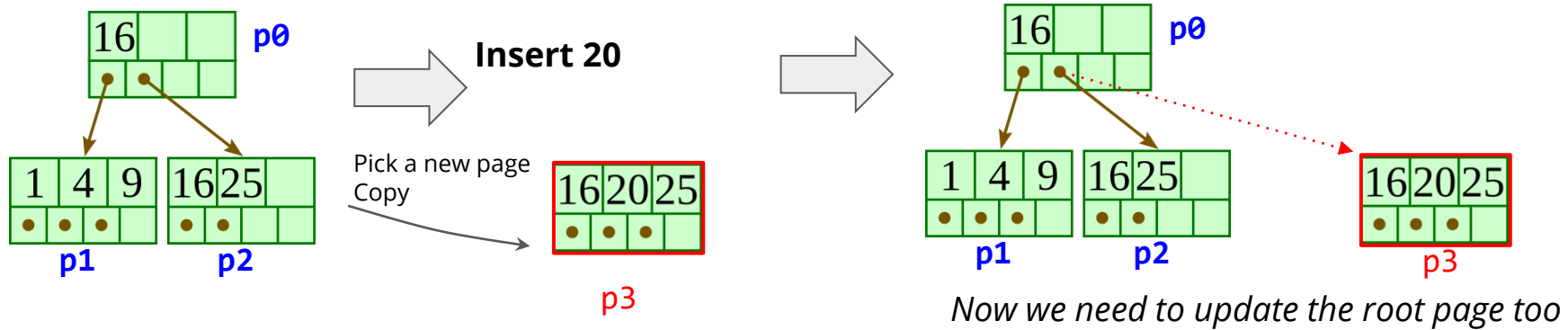


NAND pages cannot be in-place updated

Example: B+ Tree Insertions on NAND Flash



Example: B+ Tree Insertions on NAND Flash



For a simple value insertion we ended up writing 2 new pages (p3 and p4) and generating 2 old (p0 and p2) invalid pages

In general, for a tree “H” height: Read and Write “H” pages, and generates “H” invalid pages

It's the same problem what we saw in Log-Structured FSeS (**recursive update problem** or also known as **Wandering Tree problem**)

Now, what about write-heavy workloads?

Write heavy workloads on flash can be really bad

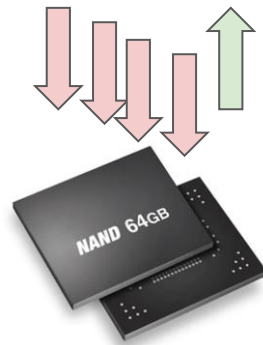
- Key-Values can be really small (32-64-128 bytes)

The best solution so far we have seen is a log (FTL, file system)

- Append small writes to a log and read from there (search)

How can we improve searching the log?

- We can build a hash table (key) $\rightarrow$ {flash offset}
 - But will need a lot of memory for the hash table
 - 8 bytes offset per key (similar to the page-level FTL challenge)
- Does not allow doing fast range-based queries and lookups



Back to the Future: LSM Trees

Log-Structured Merge (LSM) Tree data structure

Invented and optimized for HDD, why?

- Same logic as LogFS
 - Disks have fast sequential performance
 - Disks have poor random, small I/O performance
- Read/Write large chunks to disk
- Eliminates random insertions, updates and deletions

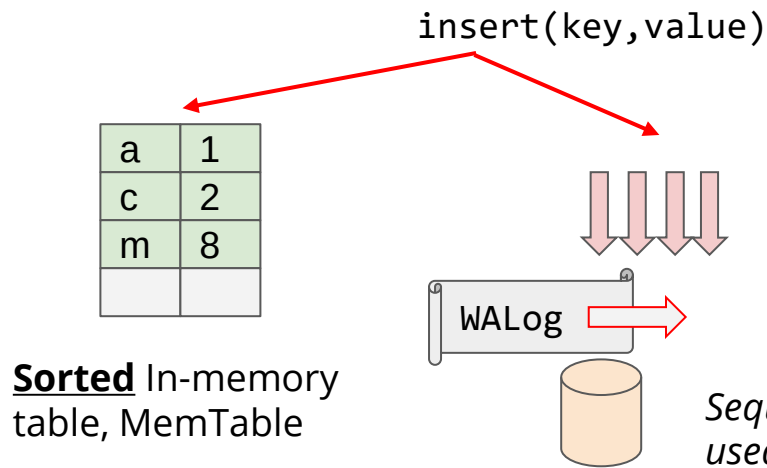
Patrick E. O'Neil, Edward Cheng, Dieter Gawlick, Elizabeth J. O'Neil:
The Log-Structured Merge-Tree (LSM-Tree). Acta Informatica 33(4):
351-385 (1996)

Very popular data structure: Bigtable, HBase, LevelDB, SQLite4, Tarantool, **RocksDB**



<https://queue.acm.org/detail.cfm?id=3220266>

LSM Tree Basics

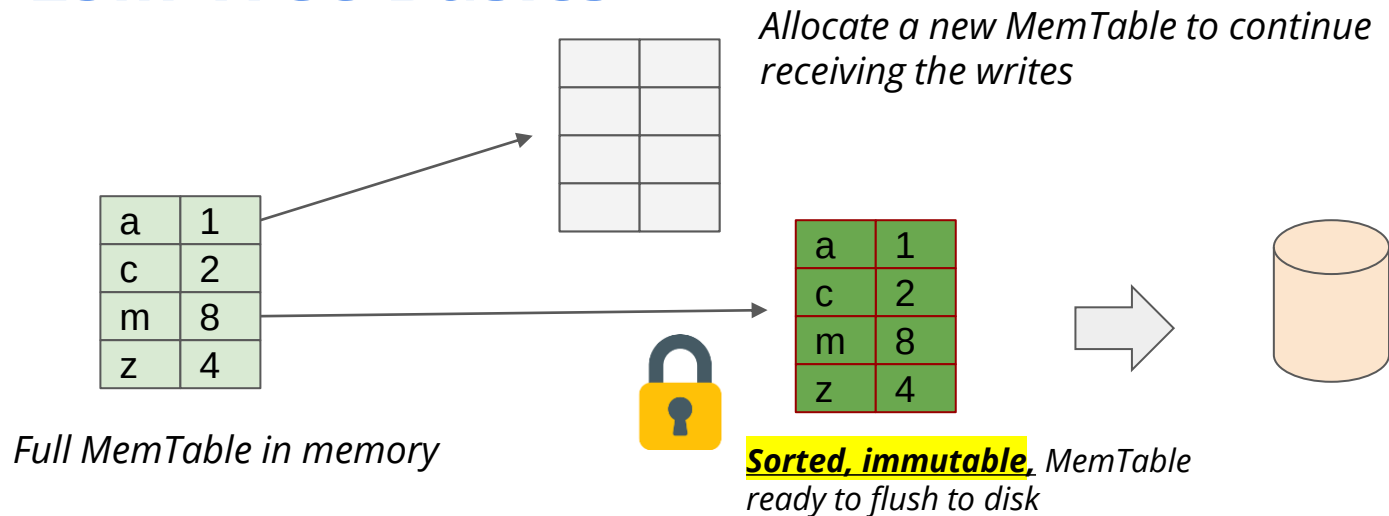


At insertion, (key,value) is

- written to the device-resident write ahead log (WAL, large sequential performance)
- Inserted in the sorted MemTable to enable fast lookup with a range based query

What happens when the in-memory data structure is full?

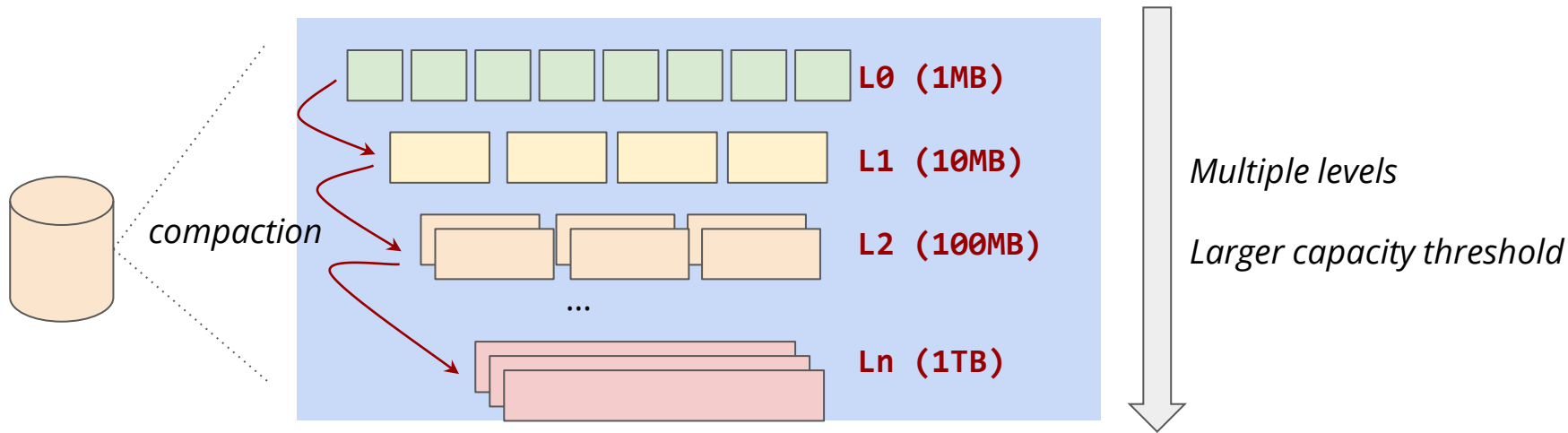
LSM Tree Basics



- Once the in-memory table is full : the MemTable is marked immutable and *flushed* to disk
- Key get() requires searching in (1) the MemTable; then (2) looking up on the disk
 - (we will see how this can be made efficient)
- If data is present in both locations, use the timestamps to reconcile which is the newest write

Challenge now is how to (a) manage and (b) search TBs of data on disk to look for a key

LSM Tree Basics



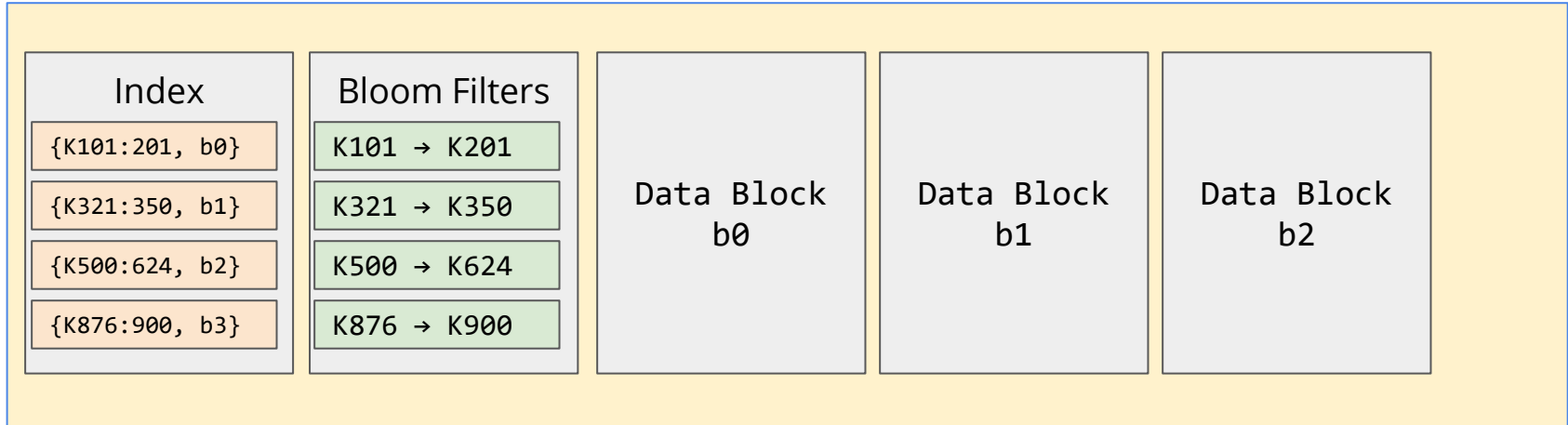
Data is stored in a multi-level, large, immutable files on the disk (no holes/gaps). Each level has a fixed size that increases as you go to the higher levels

A new table flush is written always written to **L0**

Just like in-memory table, once, a preconfigured size of file is reached, a files are level i can be merged with $(i+1)$. This process is known as **compaction**. Since files written are sorted, the compaction is essentially an N-way merge sort from level (i) to $(i+1)$

On-Disk File Format (SSTables)

Sorted String Tables (SSTables)



When searching : find a value in the index range, then check in the bloom filter

Then go fetch the “block” for reading and scan the value inside

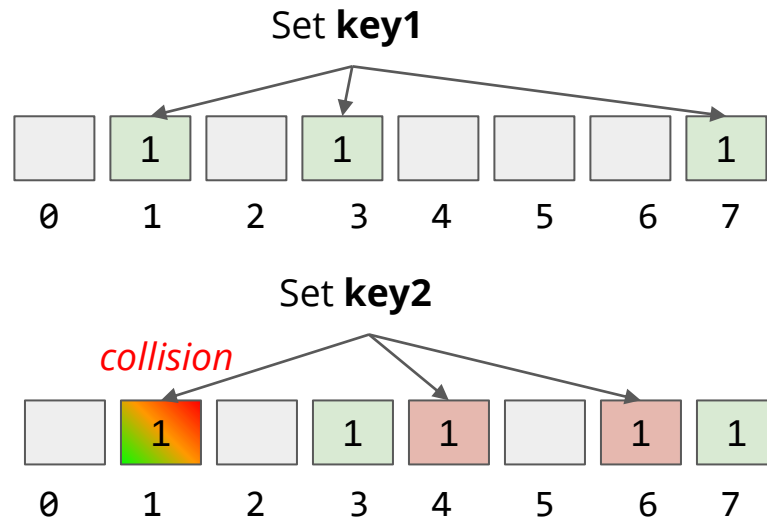
All files are immutable, hence, a delete is a new insertion with a “NULL” value at L0

Recap: Bloom Filters



Bitmap or an array (any size)

A bunch of hash function, **h1**, **h2**, **h3**



Now if we were to check the filter for (assume these bit hashes):

- lookup (**key1**) $\Rightarrow \{1, 3, 7\}$ bits // all set, key1 exists, **true positive**
- lookup (**key3**) $\Rightarrow \{1, 4, 7\}$ bits // all set, but the key3 was never set, **false positive**
- lookup (**key4**) $\Rightarrow \{0, 2, 5\}$ bits; // nope, this key was never set, always accurate!
 - **cannot have false negative!**

The rate of false positive depends upon the size of the filter (how many bits) and the quality of the hash functions

Example Compaction Process

L0

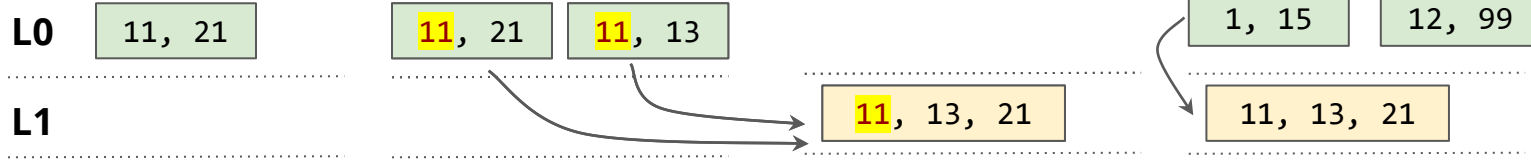
11, 21

L1

L2

Example Compaction Process

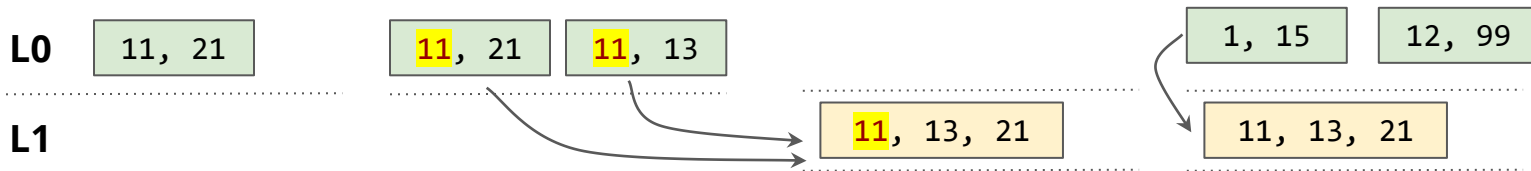
L0 can have duplicate keys in different files



Pick all files which have overlapping ranges

Example Compaction Process

L0 can have duplicate keys in different files



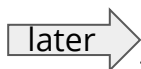
L2

Pick all files which have overlapping ranges

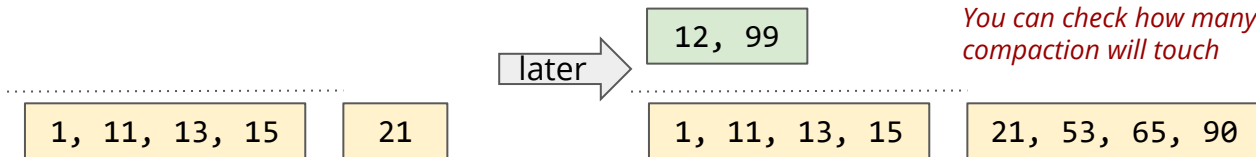
L0

L1

L2

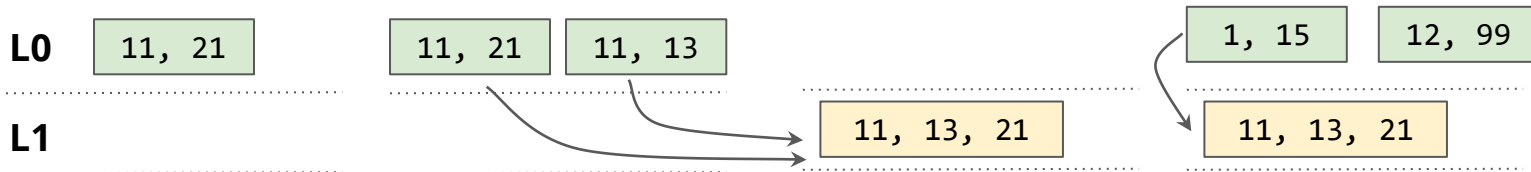


You can check how many segments this compaction will touch



Example Compaction Process

L0 can have duplicates keys in different files



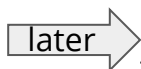
L2

Pick all files which have overlapping ranges

L0

L1

L2



You can check how many segments this compaction will touch

L0

L1

L2

2, 16, 95

1, 11, 12, 13

15, 21, 53, 65

90, 99

1, 2, 11, 12

13, 15, 16, 21

53, 65, 90, 95

99

How to Optimize for Searching Files?

Look in: (i) mutable MemTable (ii) look at all the files at L0

- L0 files can contain overlapping key ranges, hence, **all files** need to be searched at **L0**

Further down, it can be a bit simpler as

- Files at L1 onwards **do not have overlapping ranges** (they are built that way)
- Hence, for each level, only need to check the range block and the bloom filter, not need to have read the file
- Lower levels contain fresher data (e.g., data at L3 would be newer than at L5)

Also, since indexes are sorted and immutable, it support range-based queries

General LSM Considerations

What are the size threshold for each level

What are the block sizes

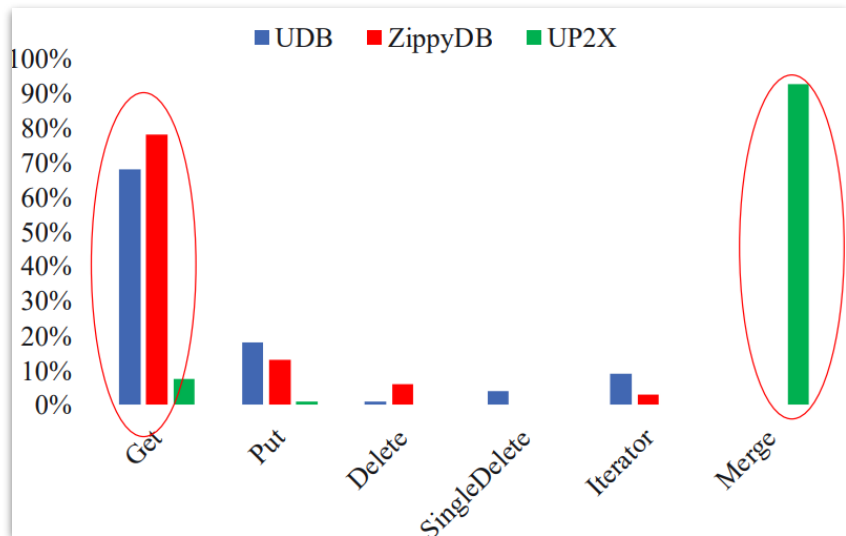
When to do compaction

- Will result in decreasing the number of files
- Which level should be compacted to which next level
- Which two files/key range to pick up for compaction (Tiered, Leveled, FIFO)
- Also: as L0 fills up the speed of writes will be stalled (in the end it will stop completely)

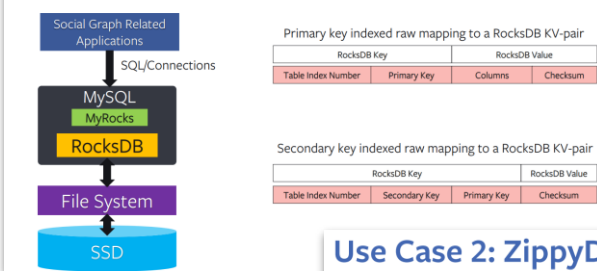
When to do garbage collection

- Deletion of old values which have been deleted
- Typically read the keys from the tree, and insert them back in the system

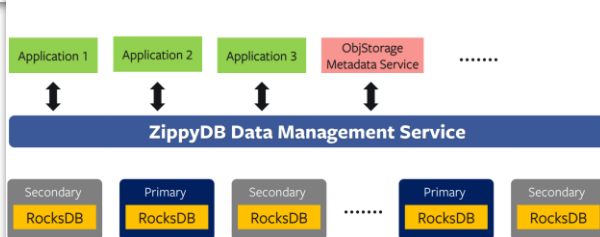
RocksDB (uses LSM tree) is very popular



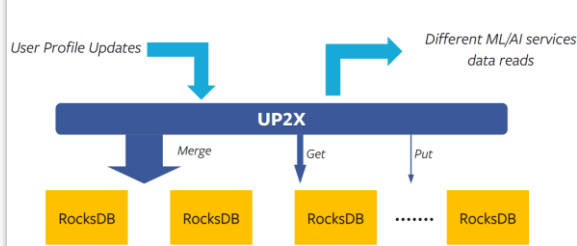
Use Case 1: UDB



Use Case 2: ZippyDB



Use Case 3: UP2X



Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook, USENIX FAST 2020.

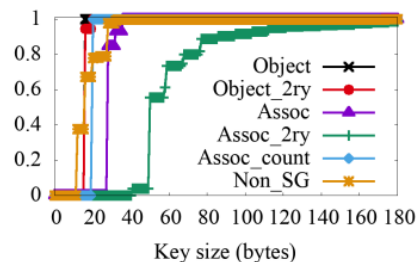
<https://www.usenix.org/conference/fast20/presentation/cao-zhichao>

Key-Value size distribution at Facebook

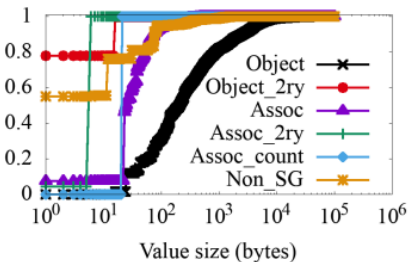
Table 2: The average key size (AVG-K), the standard deviation of key size (SD-K), the average value size (AVG-V), and the standard deviation of value size (SD-V) of UDB, ZippyDB, and UP2X (in bytes)

	AVG-K	SD-K	AVG-V	SD-V
UDB	27.1	2.6	126.7	22.1
ZippyDB	47.9	3.7	42.9	26.1
UP2X	10.45	1.4	46.8	11.6

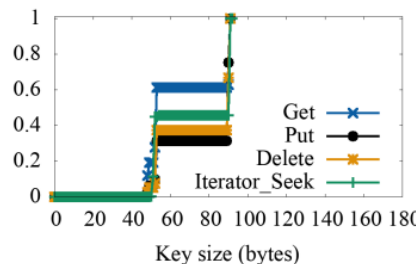
Key message: Bytes-KB ranges are very important to optimize!



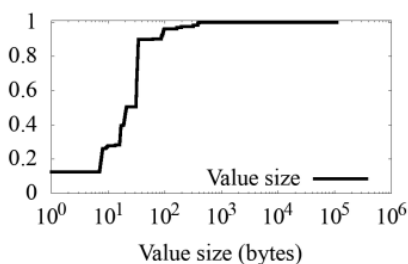
(a) UDB key size CDF



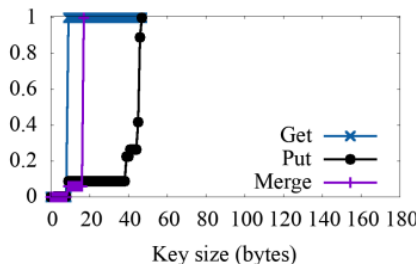
(b) UDB value size CDF



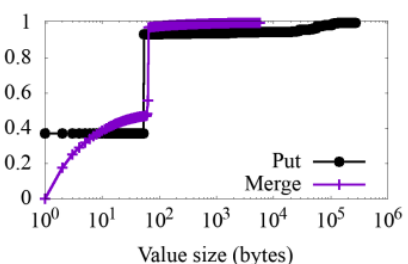
(c) ZippyDB key size CDF



(d) ZippyDB value size CDF



(e) UP2X key size CDF



(f) UP2X value size CDF

Interesting Papers SILK (2019)

SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores

Oana Balmau
University of Sydney

Florin Dinu
University of Sydney

Willy Zwaenepoel
University of Sydney

Karan Gupta
Nutanix Inc.

Ravishankar Chandhiramoorthi
Nutanix Inc.

Diego Didona
IBM Research – Zurich

Abstract

LSM-based KV stores are designed to offer good write performance, by capturing client writes in memory, and only later flushing them to storage. Writes are later compacted into a tree-like data structure on disk to improve read performance and to reduce storage space use. It has been widely documented that compactions severely hamper throughput. Various optimizations have successfully dealt with this problem. These techniques include, among others, rate-limiting flushes and compactions, selecting among compactions for maximum effect, and limiting compactions to the highest level by so-called fragmented LSMs.

In this paper we focus on latencies rather than throughput. We first document the fact that LSM KV stores exhibit high tail latencies. The techniques that have been proposed for optimizing throughput do not address this issue, and in fact in some cases exacerbate it. The root cause of these high tail latencies is interference between client writes, flushes and compactions. We then introduce the notion of an I/O scheduler for an LSM-based KV store to reduce this interference. We explore three techniques as part of this I/O scheduler: 1) opportunistically allocating more bandwidth to internal operations during periods of low load, 2) prioritizing flushes and compactions at the lower levels of the tree, and 3) pre-empting compactions.

SILK is a new open-source KV store that incorporates this

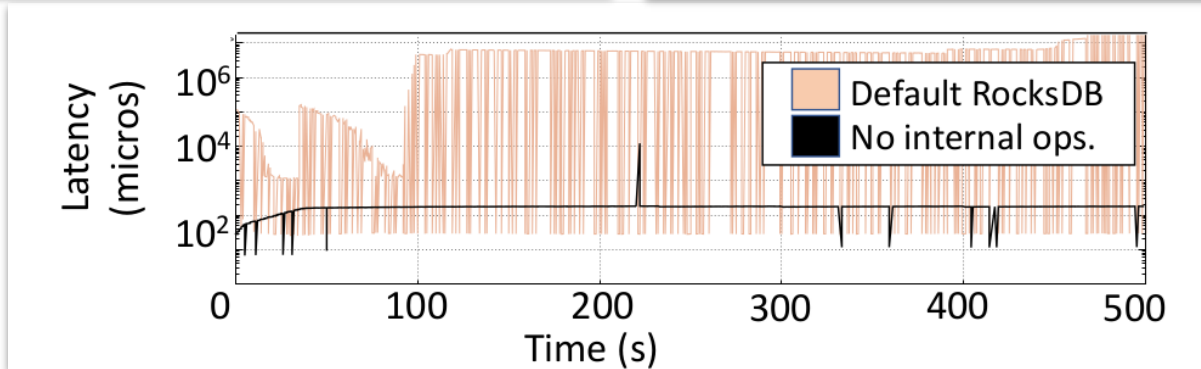
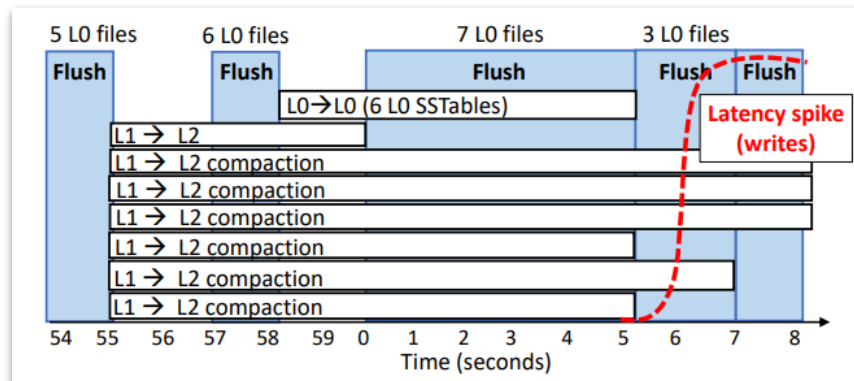
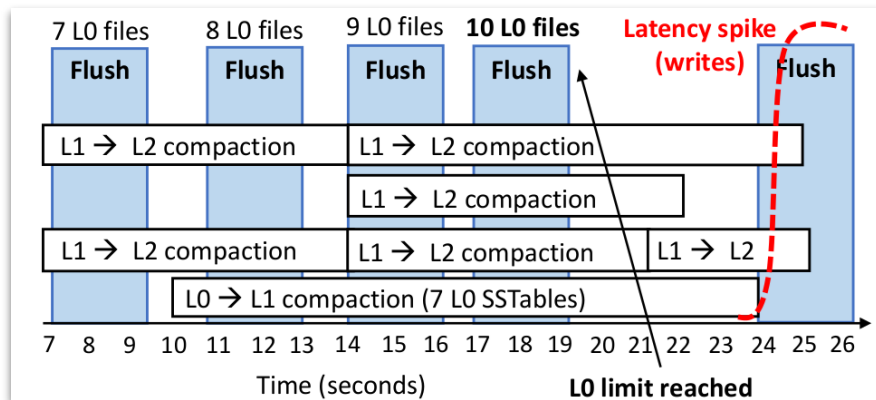
latency is especially important, because applications often exhibit high fan-out queries whose overall latency is determined by the response time of the slowest reply. Log-structured merge key-value stores (LSM KV stores), such as RocksDB [13], LevelDB [12] and Cassandra [30], are widely adopted in production environments to provide storage beyond main memory for such latency-critical applications, especially for write-heavy workloads. At Nutanix, we use LSM KV stores for storing the meta-data of our core enterprise platform, which serves thousands of customers with petabytes of storage capacity.

KV stores support a range of client operations, such as `Get()`, `Update()` and `Scan()`, to store and retrieve data. LSM KV stores strive for good update performance by absorbing updates in an in-memory buffer [36, 37]. A tree-like structure is maintained on storage. In addition to client operations, LSM KV stores implement two types of *internal operations*: *flushing*, which persists the content of in-memory buffers to disk, and *compaction*, which merges data from the lower into the higher levels of the tree.

In this paper we demonstrate that tail latencies in state-of-the-art LSM KV stores can be quite poor, especially under heavy and variable client write loads. We introduce the notion of an *I/O scheduler for LSM KV stores*. We implement this I/O scheduler in RocksDB, and we show up to two orders of magnitude improvements in tail latency.

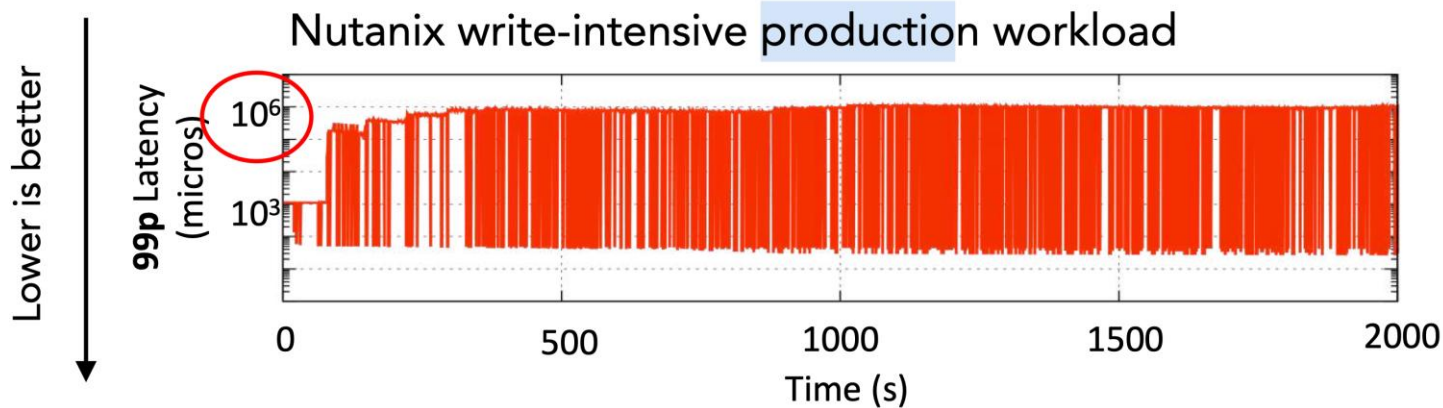
Not all LSM operations are equal

The Long Tail of LSM Trees (RocksDB)



The Long Tail of LSM Trees

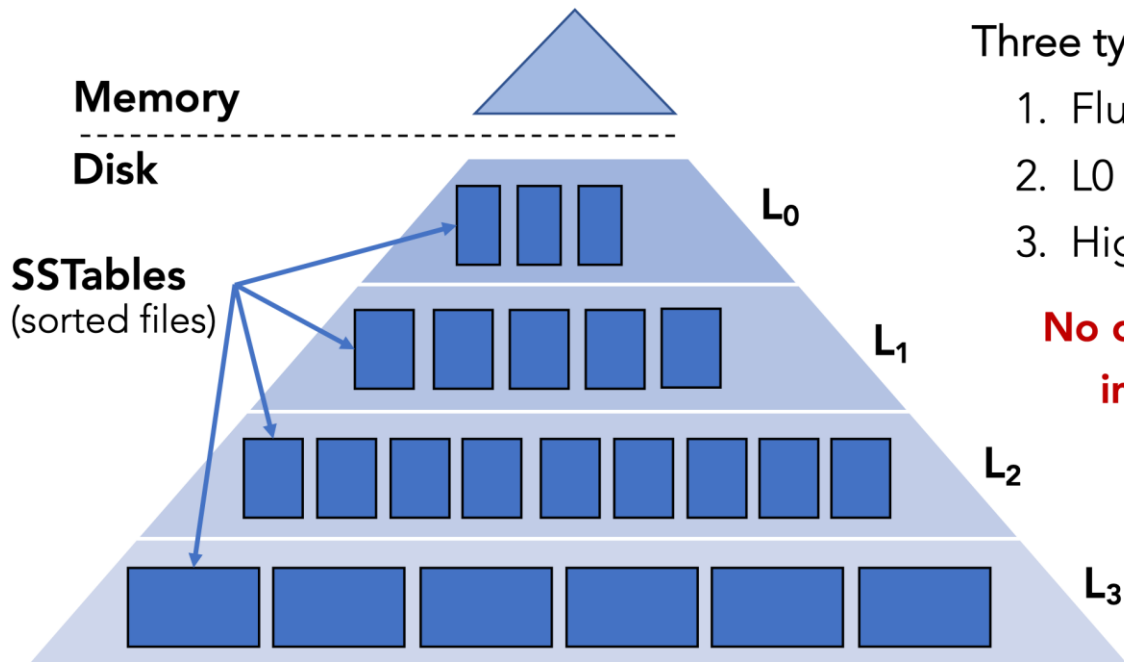
LSM KV Latency Spikes in RocksDB



Latency spikes of up to 1s in write dominated workloads.

The Long Tail of LSM Trees

LSM Internal Ops



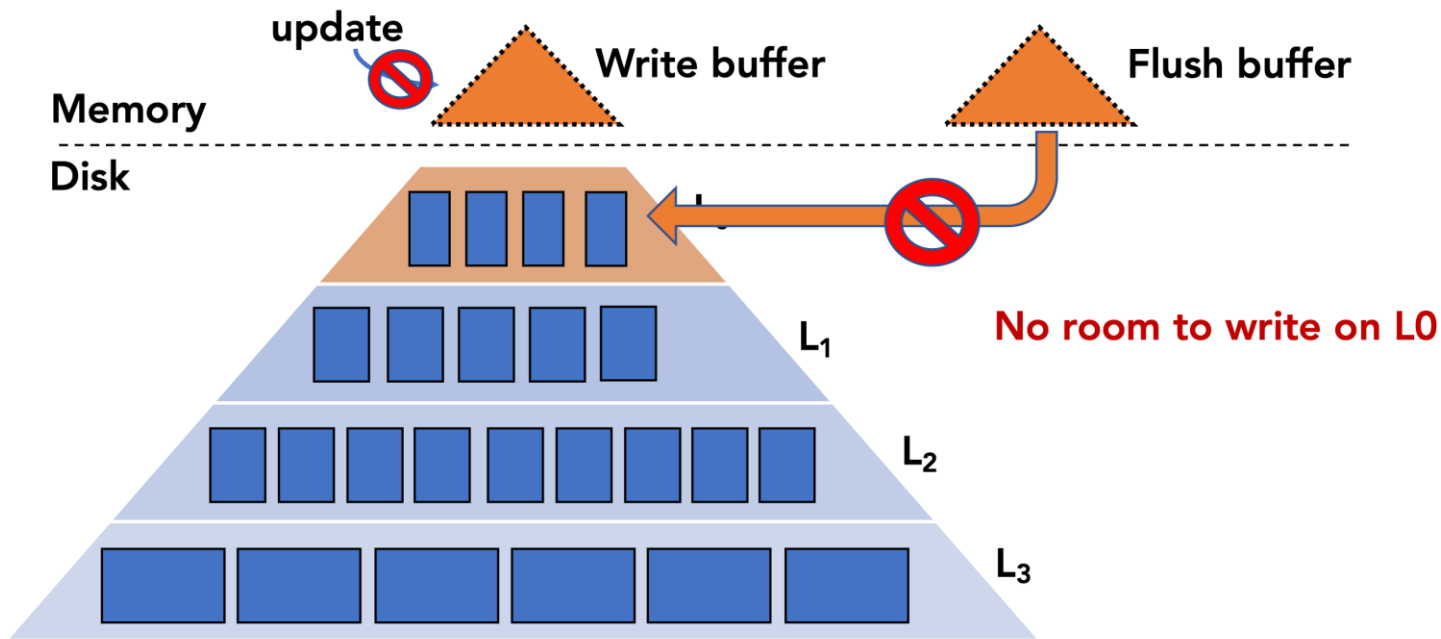
Three types of internal ops:

1. Flushing
2. $L_0 \rightarrow L_1$ compaction
3. Higher level compactions

**No coordination between
internal operations.**

The Long Tail of LSM Trees

L0 Full, Cannot Flush



The Long Tail of LSM Trees

1. Writes Blocked Because L0 is Full.

No coordination between internal ops.



Higher level compactions take over I/O.



L0 → L1 compaction is too slow.



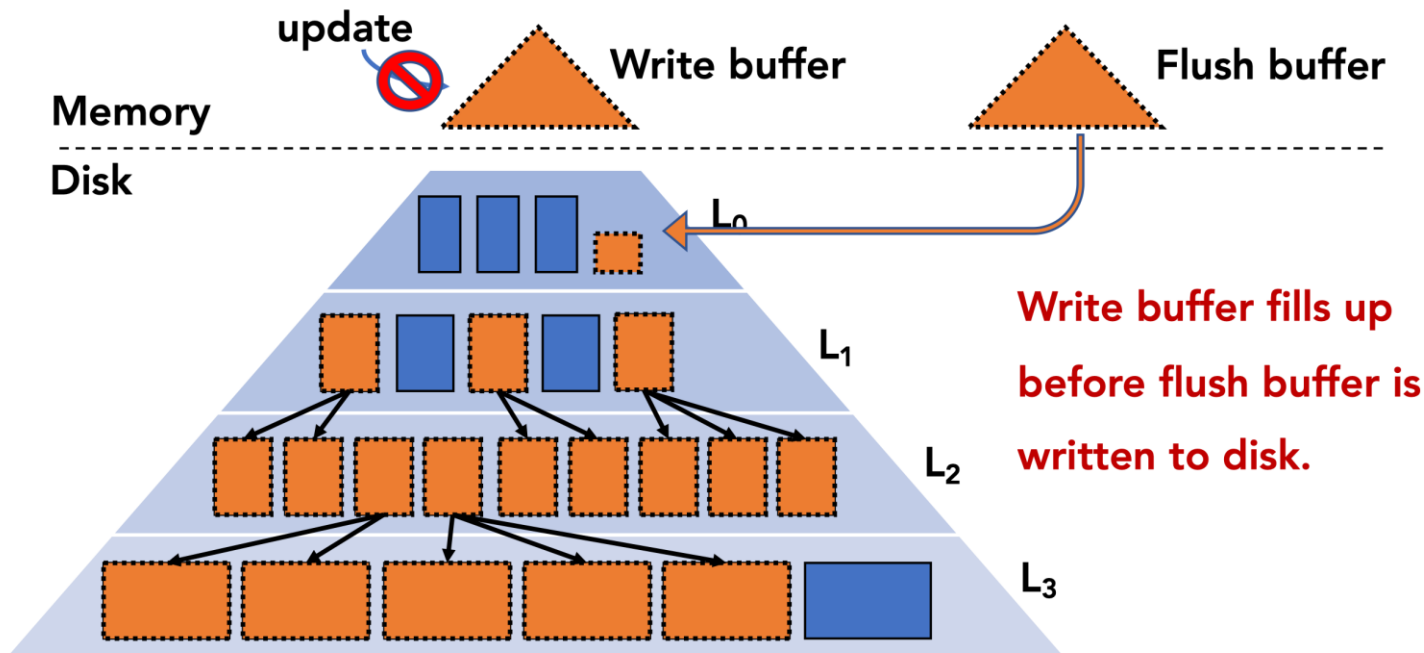
Not enough space on L0.



Cannot flush memory component.

The Long Tail of LSM Trees

Flushing is Slow



The Long Tail of LSM Trees

2. Writes Blocked Because Flushing is Slow.

No coordination between internal ops.



Higher level compactions take over I/O.



Flushing does not have enough I/O.



Flushing is very slow.



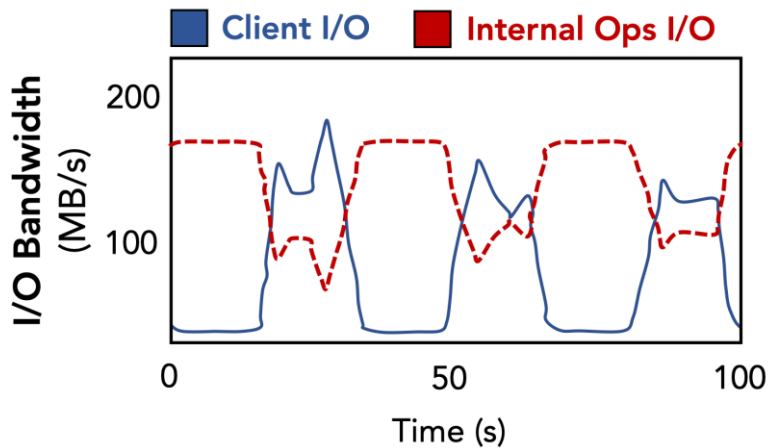
Memory component becomes full.

LSM-Tree写尾延迟过大的问题

措施1:在Client I/O请求小的时候, 增加内部操作的I/O占比(flush、compaction)

设计目标:

- (1) 确保L0层不会满;
- (2) 确保足够的IO给Flush和低层次的compaction;
- (3) 确保其它层次的compactions不会落后太多



LSM-Tree写尾延迟过大的问题



Flushing



L0 → L1 compactions



Higher level compactions

措施2:

(1) 将Flushing和L0 → L1 compaction的线程绑定在一个线程池中，其它高层次的compaction则在其它线程池中；

(2) 将I/O带宽以及CPU等资源优先调度给Flush和L0 → L1 compactions.

(3) L0 → L1和flush线程可以抢占高层次的I/O带宽

WiscKey: Separating Keys from Values in SSD-Conscious Storage

WiscKey: Separating Keys from Values in SSD-Conscious Storage

Lanyue Lu, Thanumalayan Sankaranarayanan Pillai,
Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau

University of Wisconsin, Madison

Abstract

We present WiscKey, a persistent LSM-tree-based key-value store with a performance-oriented data layout that separates keys from values to minimize I/O amplification. The design of WiscKey is highly SSD optimized, leveraging both the sequential and random performance characteristics of the device. We demonstrate the advantages of WiscKey with both microbenchmarks and YCSB workloads. Microbenchmark results show that WiscKey is $2.5\times$ – $111\times$ faster than LevelDB for loading a database and $1.6\times$ – $14\times$ faster for random lookups. WiscKey is faster than both LevelDB and RocksDB in all six YCSB workloads.

1 Introduction

Persistent key-value stores play a critical role in a variety of modern data-intensive applications, including web indexing [16, 48], e-commerce [24], data deduplication [7, 22], photo stores [12], cloud data [32], social networking [9, 25, 51], online gaming [23], messaging [1, 29], software repository [2] and advertising [20]. By enabling efficient insertions, point lookups, and range queries, key-value stores serve as the foundation for this growing group of important applications.

For write-intensive workloads, key-value stores based on Log-Structured Merge-Trees (LSM-trees) [43] have become the state of the art. Various distributed and local stores built on LSM-trees are widely deployed in large-scale production environments, such as BigTable [16] and LevelDB [48] at Google, Cassandra [33], HBase [29] and RocksDB [25] at Facebook, PNUTS [20] at Yahoo!, and Riak [4] at Basho. The main advantage of LSM-

throughout its lifetime; as we show later (§2), this I/O amplification in typical LSM-trees can reach a factor of $50\times$ or higher [39, 54].

The success of LSM-based technology is tied closely to its usage upon classic hard-disk drives (HDDs). In HDDs, random I/Os are over $100\times$ slower than sequential ones [43]; thus, performing additional sequential reads and writes to continually sort keys and enable efficient lookups represents an excellent trade-off.

However, the storage landscape is quickly changing, and modern solid-state storage devices (SSDs) are supplanting HDDs in many important use cases. As compared to HDDs, SSDs are fundamentally different in their performance and reliability characteristics; when considering key-value storage system design, we believe the following three differences are of paramount importance. First, the difference between random and sequential performance is not nearly as large as with HDDs; thus, an LSM-tree that performs a large number of sequential I/Os to reduce later random I/Os may be wasting bandwidth needlessly. Second, SSDs have a large degree of internal parallelism; an LSM built atop an SSD must be carefully designed to harness said parallelism [53]. Third, SSDs can wear out through repeated writes [34, 40]; the high write amplification in LSM-trees can significantly reduce device lifetime. As we will show in the paper (§4), the combination of these factors greatly impacts LSM-tree performance on SSDs, reducing throughput by 90% and increasing write load by a factor over 10. While replacing an HDD with an SSD underneath an LSM-tree does improve performance, with current LSM-tree technology, the SSD's true potential goes largely unrealized.

LSM has Read and Write Amplifications

We briefly referenced that reading performance on LSM can be problematic

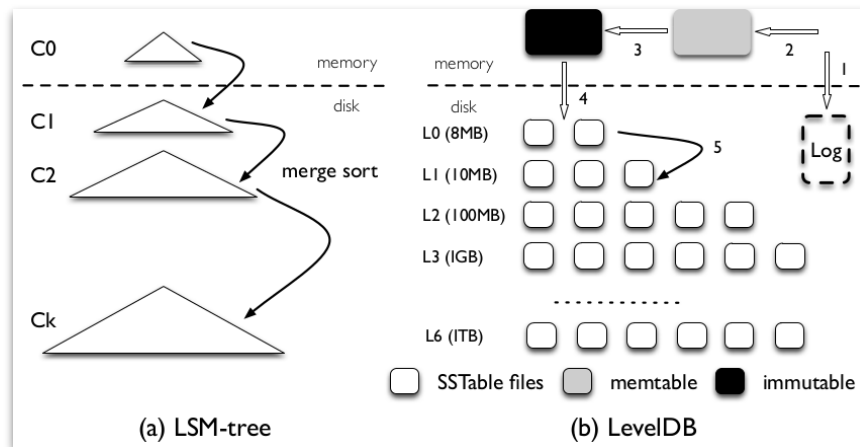
Any guesses why?

What was the read path order?

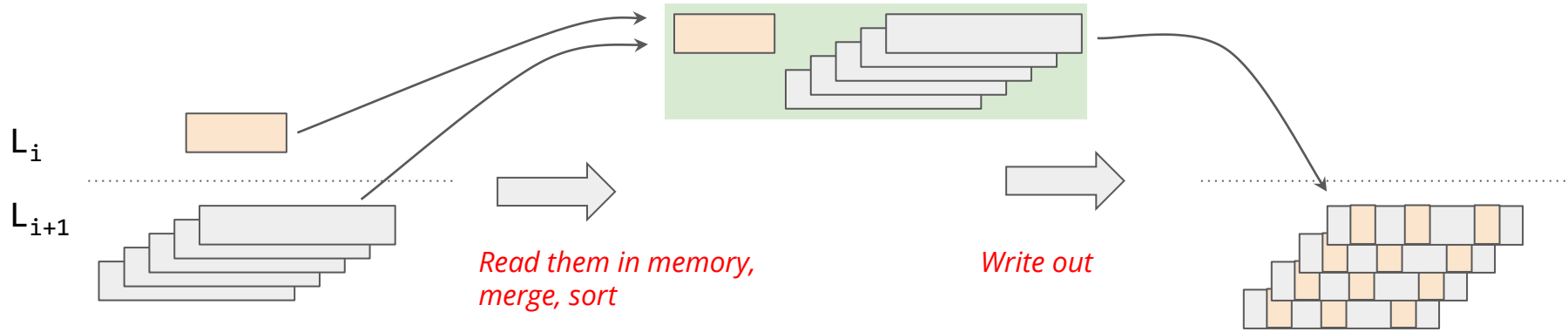
- MemTable → L0 → L1 ... L6 (here)

So, if you were to read simple 1 byte key-value, how much data you have to read before you can find a 1 byte result?

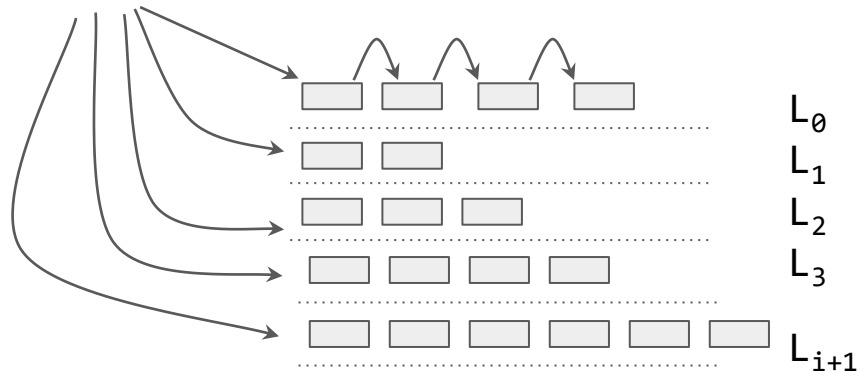
We have looked this type of problem before in the FTL for writes (**recall**: write-amplification)



LSM has Read and Write Amplifications



read,lookup



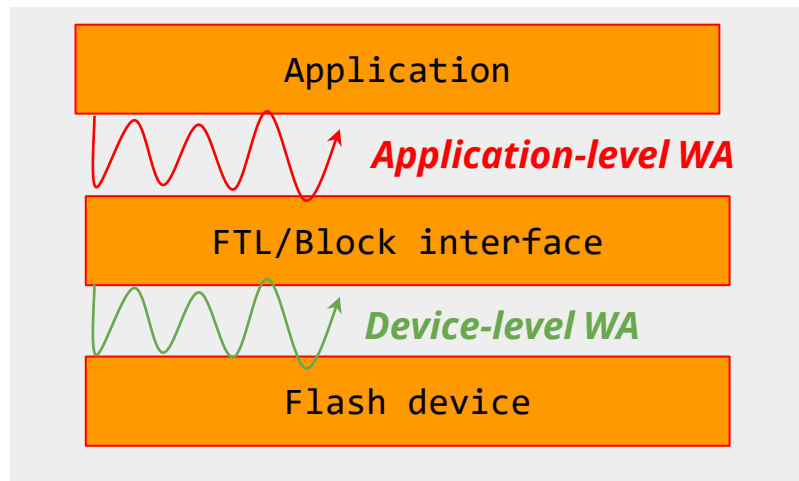
Analysis : Write/Read Amplification (RA/WA)

Compaction can result in

- Reading “n” times data from the next level to merge from the current level
 - For LevelDB this is 10x between levels
 - For 6 levels, it could be 50x

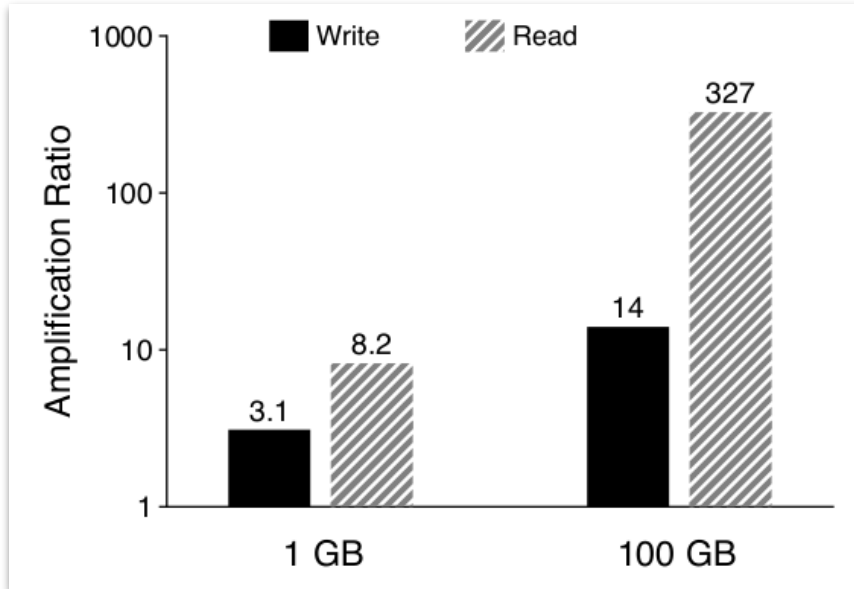
Reading can result in

- Reading “n” files on L0 and then 1 file on following level
 - LevelDB, 8 files (at L0) + 6 files (L1-L6) = 14 files
 - Within the file we need to read the “index” + “bloom filter” + data block
 - For Level-DB index (16kB), bloom (4kB) + data (4kB)
 - So, if we are looking for a 1kB file: 14 files x (24 kb) = 336 kb $\Rightarrow$ 336x **RA**
 - Determined by how many files do you have to touch and read to find a value



LSM Trees trade high “amplification” for having “sequential performance” $\rightarrow$ Why does this design make sense?

Quantify and Justify



Key size: 16 bytes, value size : 1024 bytes

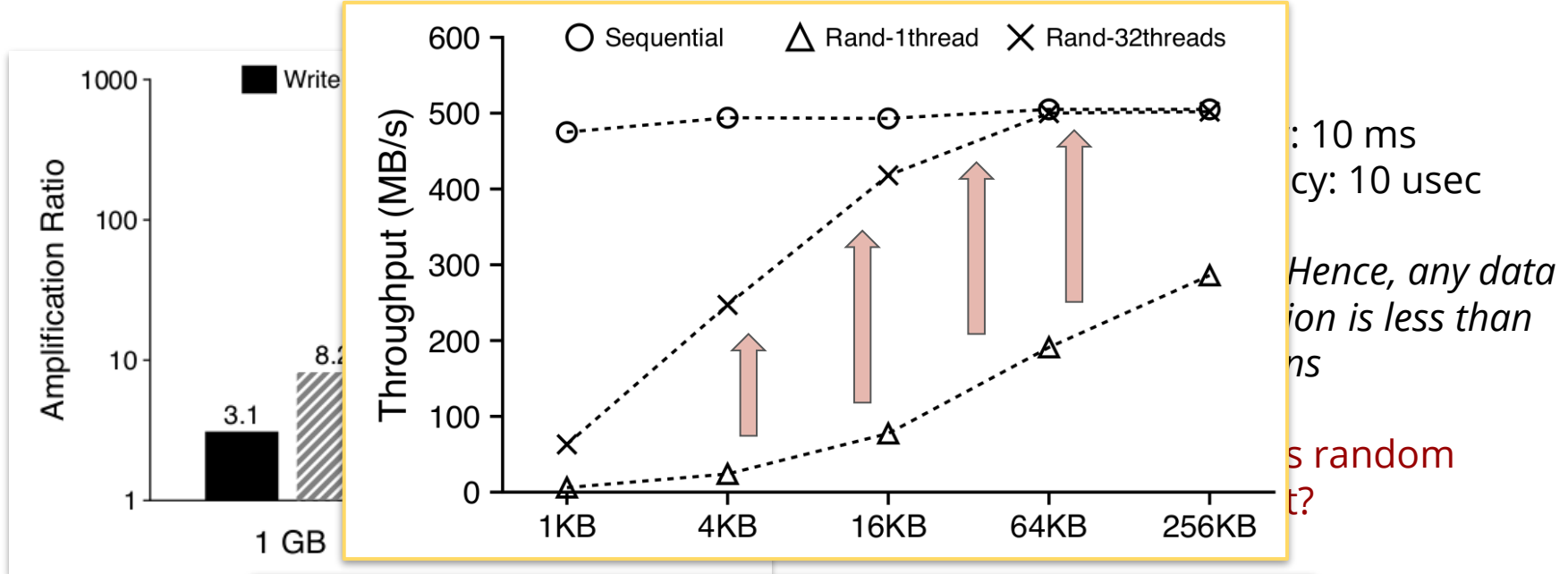
Justification for HDD

- Random 1kB latency: 10 milli-sec
- Sequential 1kB latency: 10 micro-sec

Ratio is seq:rand **1:1000**. Hence, any data structure where amplification is less than 1000, sequential access wins

On SSD? Are sequential vs random accesses are 1:1000 apart?

Quantify and Justify

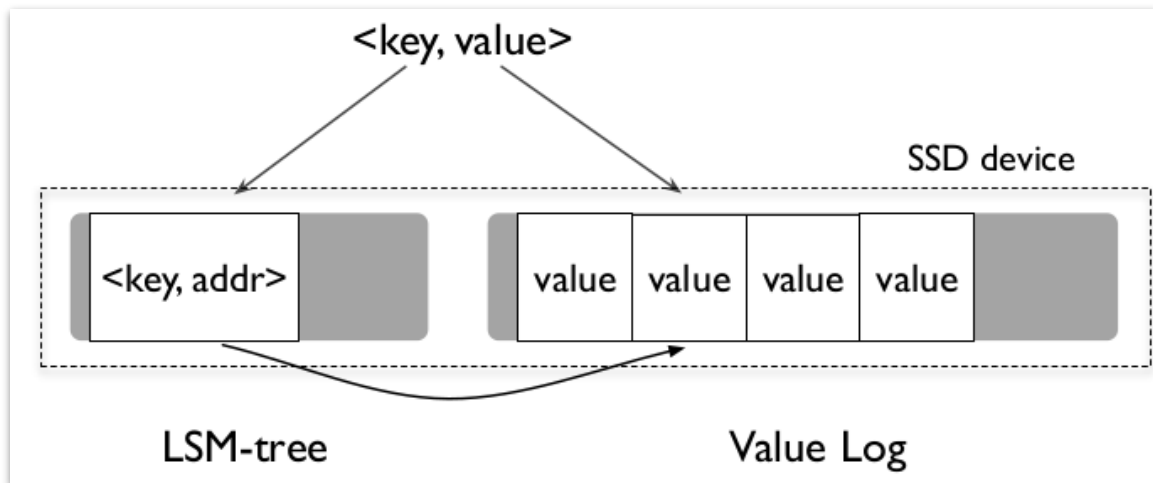


- There exists a gap between random and sequential performance, but
- Not for large values
 - The gap can be closed by issuing multiple parallel requests

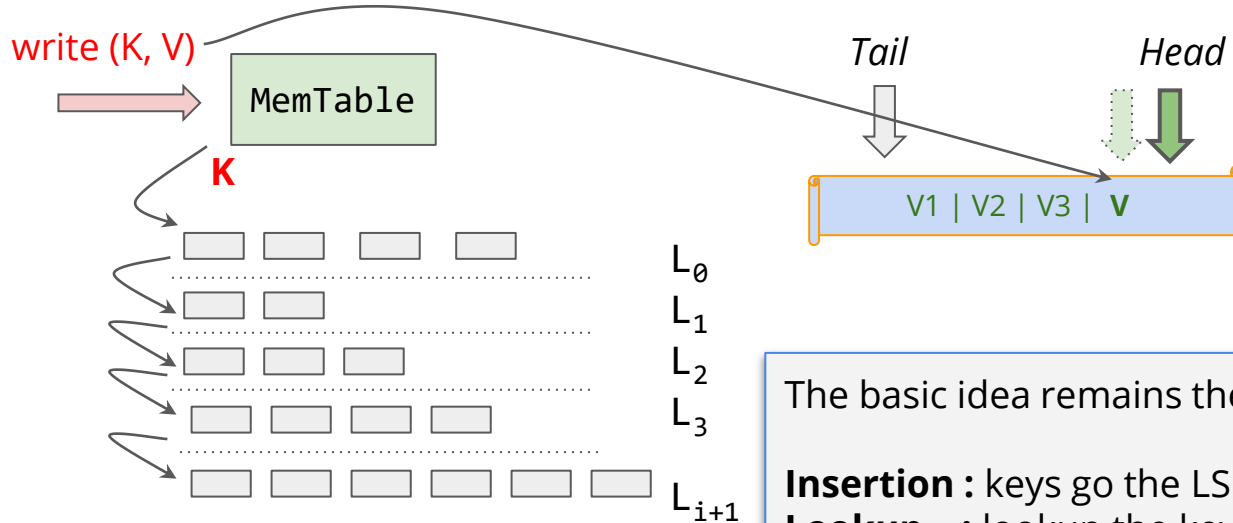
What does WiscKey Proposes

Key Idea: separate keys from the values

- Maintain keys in the LSM tree
- Maintain value in a sequential append value log



Key-Value Insertion and Lookup



The basic idea remains the same

Insertion : keys go the LSM tree, values to the log

Lookup : lookup the key in the LSM tree, then read the offset from the log

For **range-based queries**, the log can be read in parallel

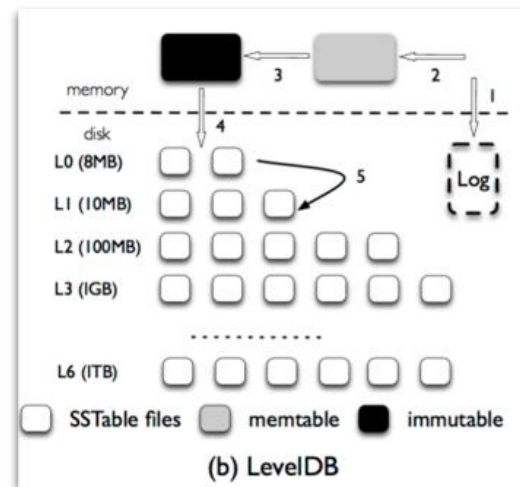
WiscKey: LSM Tree made out of Keys

What advantages a key-only LSM tree brings

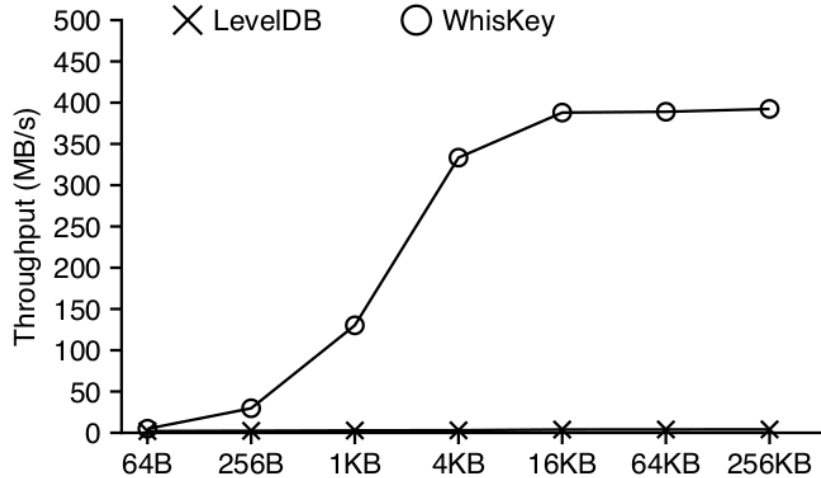
- [with assumptions] keys are small and values are big
- Much improved write-amplification
 - Before WA was: ~10-50x
 - Now $(10 \times \text{key_size}) + \text{value_size} / (\text{key} + \text{value size})$
 - E.g., $(10 \times 16 + 1024) / (1024 + 16) = \mathbf{1.14}$ (not 10x)
 - Worse case : $(50 \times 16 + 1024) / (1024+16) = \mathbf{1.76}$ (not 50x)
- Lower write amplification means longer device life time

Also, the size of the tree can be small (small keys)

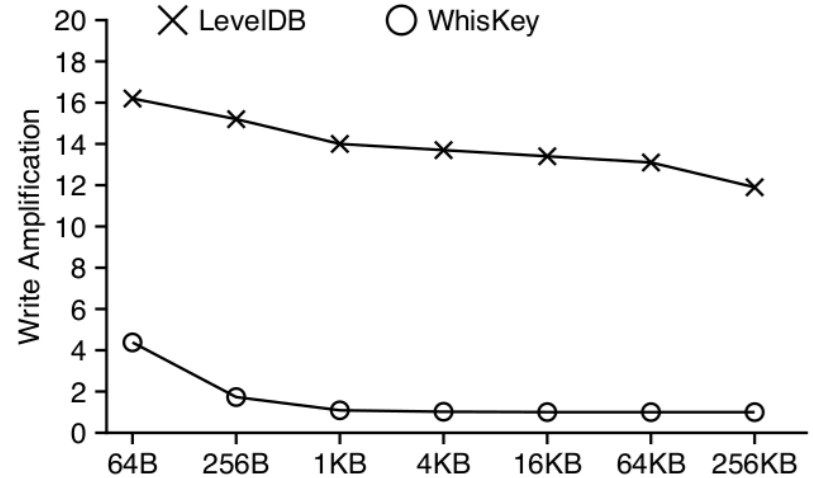
- Less levels than a comparable key-value LSM tree
- Small tree can be cached in the memory for fast lookups



WiscKey: Performance



Key: 16B, Value: 64B to 256KB

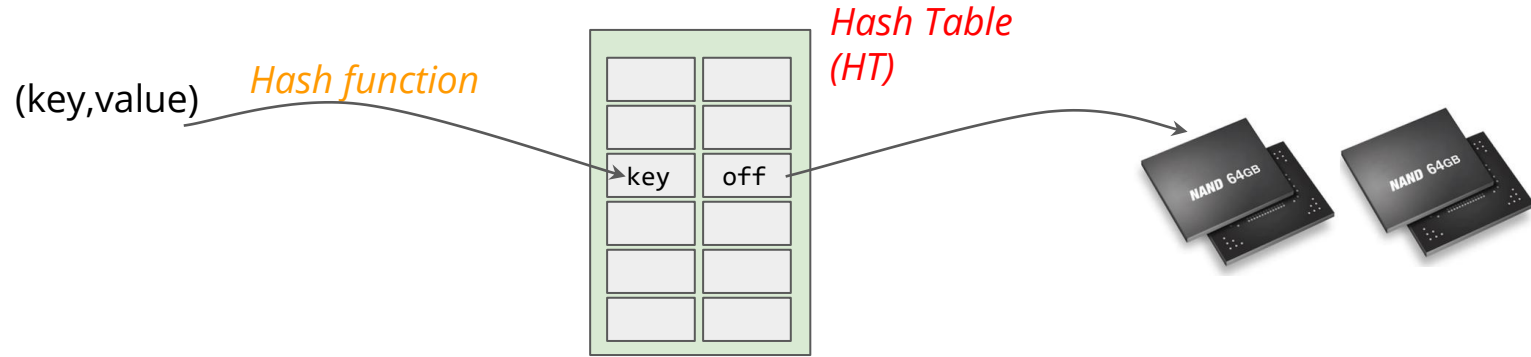


Key: 16B, Value: 64B to 256KB

LevelDB is at 2-4MB/sec whereas WiscKey is at 350 MB/sec (46-111x)

Significant reduction in the WA factor

Hash Tables on Flash



This simple hash table based schema works, but it needs to deal with

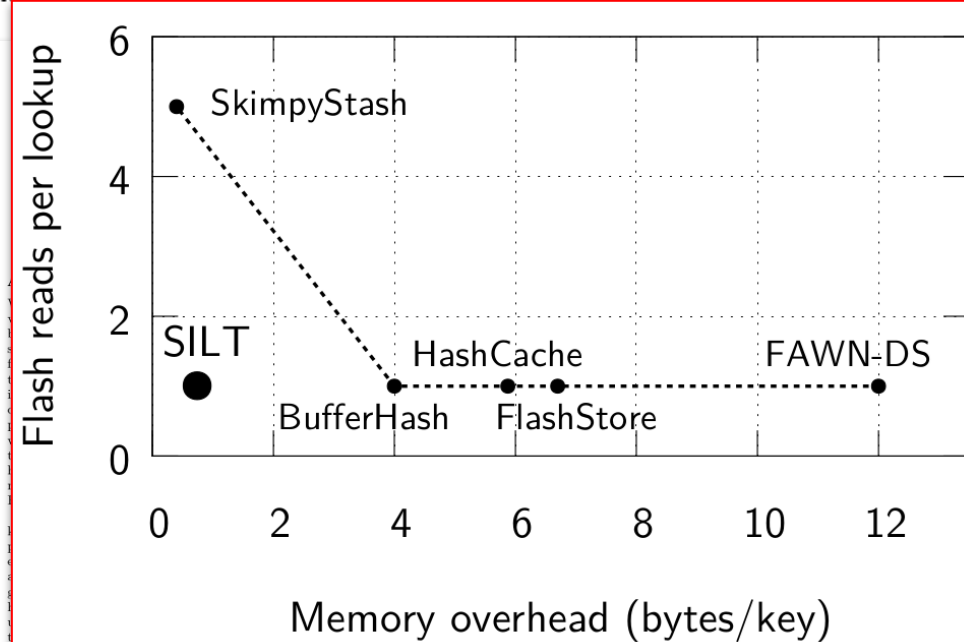
- Small writes (multiple writes must be packed together)
- Can do fast get and put, but no range-based queries (without additional indexes)
- Trade off {DRAM size of the HT} $\leftarrow \rightarrow$ {number of I/O operations}
 - The same tradeoff as FTL design, how much memory do we need to store a hash table with 1 TB of values
 - Can store the table in flash itself, to decrease the memory size, then multiple I/O

Alternate Hash Table Designs (see the backup slides)

SkimpyStash: RAM Space Skimpy Key-Value Store on Flash-based Storage

ABSTRACT

We present SkimpyStash, a RAM space efficient key-value store designed for high throughput server applications. The distinguishing feature is the design goal of extremely low RAM footprint (0.5 byte per key-value pair, which is smaller than most prior designs). SkimpyStash uses a hash table index key-value pairs stored in a log-structured flash. To break the barrier of a flash pointer (say, 4 bytes) overhead per key, it "moves" most of the pointer information from RAM to flash itself, resolving hash table collisions using linear probing. It uses a single key to resolve (collide) to the same bucket, and (ii) storing the pointer in the beginning record of the chain on flash. This design results in a small number of flash reads per lookup. Two further techniques improve performance: (iii) two-choice based bucket sizes (hence, chain lookup times), and a bloom filter in each bucket in RAM to disambiguate the choice during bucket selection procedure to pack bucket chain records. This design results in a small number of flash reads per lookup. The bucket size is the critical design parameter for making a continuum of tradeoffs between usage and low lookup latencies. Our evaluation on server platforms with real-world data center workloads shows that SkimpyStash provides throughputs from 100,000 get-set operations/sec.



ate the design of FlashStore on traditional and low power server platforms. FlashStore outperforms BerkeleyDB by up to 60x on throughput (ops/sec), up to 50x on energy efficiency (ops/Joule), and up to 85x on cost efficiency (ops/sec/dollar) on the evaluated datasets.

FlashStore, a high performance key-value storage system using flash as a cache between RAM and hard disk. When a key-value blob is written, it is sequentially logged in flash. A specialized RAM-space efficient hash table index using a variant of cuckoo hashing [32] and compact key signatures is used to index the key-value blobs stored in flash mem-

SILT: A Memory-Efficient, High-Performance Key-Value Store

Huotao Lin¹, Bin Fan¹, David G. Andersen¹, Michael Kaminsky²

¹Carnegie Mellon University, ²Intel Labs

Metric	2008 → 2011	Increase
CPU transistors	731 → 1,170 M	60 %
DRAM capacity	0.062 → 0.153 GB/S	147 %
Flash capacity	0.134 → 0.428 GB/S	219 %
Disk capacity	4.92 → 15.1 GB/S	207 %

Table 1: From 2008 to 2011, flash and hard disk capacity increased much faster than either CPU transistor count or DRAM capacity.

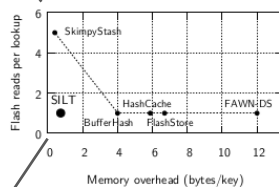


Figure 1: The memory overhead and lookup performance of SILT and the recent key-value stores. For both axes, smaller is better.

e-commerce platforms [21], data deduplication [1, 19, 20], picture stores [7], web object caching [4, 30], and more.

To achieve low latency and high performance, and make best use of limited I/O resources, key-value storage systems require efficient indexes to locate data. As one example, Facebook engineers recently created a new key-value storage system that makes aggressive use of DRAM-based indexes to avoid the bottleneck caused by multiple disk operations when reading data [7]. Unfortunately, DRAM is up to 8X more expensive and uses 25X more power per bit than flash, and as Table 1 shows, it is growing more slowly than the capacity of

The RUM Conjecture

Read overheads (RO)

= total read / user read

Update overheads (UO)

= total write / user (or logical) write

Space/Memory overheads (MO)

= total space / data space

"An access method that can set an upper bound for two out of the read, update, and memory overheads, also sets a lower bound for the third overhead."

Or: all three can not be simultaneously optimized to their optimal value.

Question: what is an optimal value for them?

Designing Access Methods: The RUM Conjecture

Manos Athanassoulis^{*} Michael S. Kester^{*} Lukas M. Maas^{*} Radu Stoica[†]

Stratos Idreos^{*} Anastasia Ailamaki[‡] Mark Callaghan[§]

^{*}Harvard University [†]IBM Research, Zurich [‡]EPFL, Lausanne [§]Facebook, Inc.

ABSTRACT

The database research community has been building methods to store, access, and update data for more than four decades. Throughout the evolution of the structures and techniques used to access data, access methods adapt to the ever changing hardware and workload requirements. Today, even small changes in the workload or the hardware lead to a redesign of access methods. The need for new designs has been increasing as data generation and workload diversification grow exponentially, and hardware advances introduce increased complexity. New workload requirements are introduced by the emergence of new applications, and data is managed by large systems composed of more and more complex and heterogeneous hardware. As a result, it is increasingly important to develop application-aware and hardware-aware access methods.

The fundamental challenges that every researcher, systems architect, or designer faces when designing a new access method are how to minimize, i) read times (R), ii) update cost (U), and iii) memory (or storage) overhead (M). In this paper, we conjecture that when optimizing the read-update-memory overheads, optimizing in any two areas negatively impacts the third. We present a simple model of the RUM overheads, and we articulate the *RUM Conjecture*. We show how the RUM Conjecture manifests in state-of-the-art access methods, and we envision a trend toward RUM-aware access methods for future data systems.

1. INTRODUCTION

Chasing Access Paths. Picking the proper physical design (through static autotuning [14], online tuning [13], or adaptively [31]) and access method [27, 49] have been key research challenges of data management systems for several decades. The way we physically organize data on storage devices (disk, flash, memory, caches) defines and restricts the possible ways that we can read and update it. For example, when data is stored in a heap file without an index, we have to perform costly scans to locate any data we are interested in. Conversely, a tree index on top of the heap file, uses additional space in order to substitute the scan with a more lightweight index probe. Over the years, we have seen a plethora of exciting and innovative proposals for data structures and algorithms, each

one tailored to a set of important workload patterns, or for matching critical hardware characteristics. Applications evolve rapidly and continuously, and at the same time, the underlying hardware is diverse and changes quickly as new technologies and architectures are developed [11]. Both trends lead to new challenges when designing data management software.

The RUM Tradeoff. A close look at existing proposals on access methods¹ reveals that each is confronted with the same fundamental challenges and design decisions again and again. In particular, there are three quantities and design parameters that researchers always try to minimize: (1) the read overhead (R), (2) the update overhead (U), and (3) the memory (or storage) overhead (M), henceforth called the *RUM overheads*. Deciding which overhead(s) to optimize for and to what extent, remains a prominent part of the process of designing a new access method, especially as hardware and workloads change over time. For example, in the 1970s one of the critical aspects of every database algorithm was to minimize the number of random accesses on disk; fast-forward 40 years and a similar strategy is still used, only now we minimize the number of random accesses to main memory. Today, different hardware runs different applications but the concepts and design choices remain the same. New challenges, however, arise from the exponential growth in the amount of data generated and processed, and the wealth of emerging data-driven applications, both of which stress existing data access methods.

The RUM Conjecture: Read, Update, Memory – Optimize Two at the Expense of the Third. An ideal solution is an access method that always provides the lowest read cost, the lowest update cost, and requires no extra memory or storage space over the base data. In practice, data structures are designed to compromise between the three RUM overheads, while the optimal design depends on a multitude of factors like hardware, workload, and user expectations.

We analyze the lower bounds for the three overheads (read - update - memory) given an access method which is perfectly tailored for minimizing each overhead and we show that such an access method will impact the rest of the overheads negatively. We take this observation a step further and propose the *RUM Conjecture*: designing access methods that set an upper bound for two of the RUM overheads, leads to a hard lower bound for the third overhead which cannot be further reduced. For example, in order to minimize the cost of updating data, one would use a design based on differential structures, allowing many queries to consolidate updates and avoid the cost of reorganizing data. Such an approach, however, increases the space overhead and hinders read cost as now queries need to merge any relevant pending updates during processing. Another example is that the read cost can be minimized by

¹Access methods: algorithms and data structures for organizing and accessing data [27].

Examples:

Minimizing RO: an indexed array (1.0)

{1, v1} {3, v2} => store in a sparse array



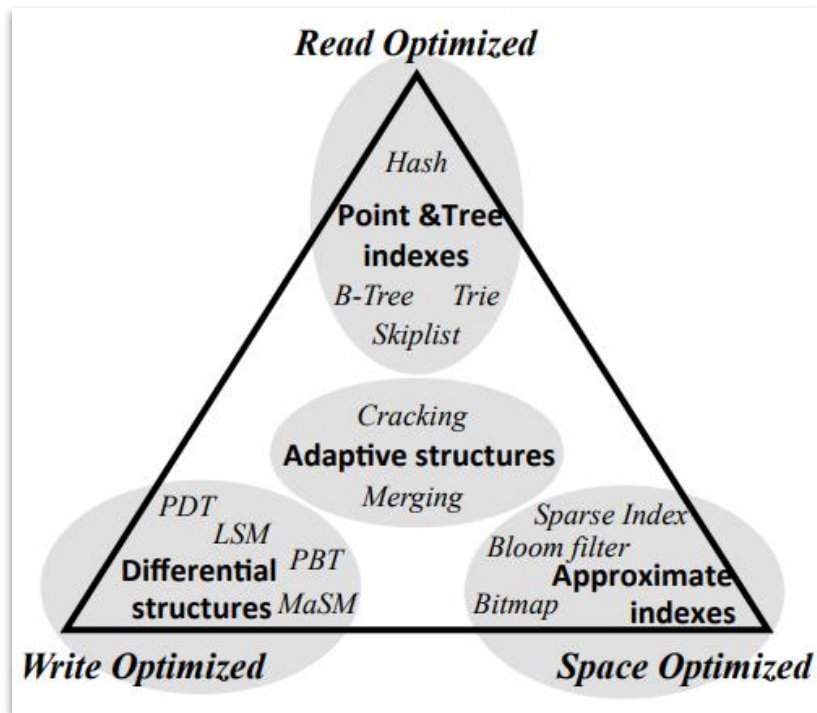
WO = 2.0 (why 2.0?), MO = $O(\infty)$ (why infinity?)

Minimizing UO: append log with diffs updates

RO = $O(\infty)$, and MO = $O(\infty)$

Minimizing MO: just store the raw user data (1.0) as a sequence

RO = $O(N)$, and UO = $O(1.0)$



Implications for Indexing structures

The RUM Conjecture: Need for efficient data-structure designs

- Read-heavy, write-heavy, mixed, range scans, concurrency, batch operations
- Modeling, statistics, and analysis

Index	Search	Insertion deletion	Space	Experimental evaluation
SCATTERED LOGGING				
BFTL [149, 151]	$h * c$	$2(\frac{1}{M-1} + \tilde{N}_{split} + \tilde{N}_{merge/rotate})$	$n * c + B$	B-tree
WOBF [51]	–	–	–	BFTL, IBSF
SCATTERED LOGGING & NODE MODIFICATION				
FlashB-tree [73]	$(h, h * c)$	$\frac{1}{n_{in}}(\frac{2}{3} * \tilde{N}_{split})$	$n + B$	BFTL, IBSF
NODE MODIFICATION				
uB+tree [141]	–	–	–	B+tree
BF-tree [7]	$h + \lfloor p_{ip} * n_{pl} \rfloor_{sr}$	–	$n * n_{pl}$	B+tree, FD-tree, hashing
IN MEMORY BUFFERING				
IBSF [88]	h	$\frac{1}{n_{in}}(\tilde{N}_{split} + \tilde{N}_{merge/rotate})$	$n + B$	BFTL
RBFTL [152]	–	–	–	B-tree
LU B+tree [116]	–	–	–	B+tree
TNC [59]	–	–	–	–
AS B-tree [123]	–	–	–	B+tree, BFTL, LA-tree
FLASH BUFFERING				
FD-tree [98, 99]	$\log_k n$	$\lceil \frac{k}{f-k} \log_k n \rceil_{srw}$	$c * n$	B+tree, BFTL, LSM-tree
FD+tree, FD+FC [139]	$\log_{\gamma} \frac{n}{\kappa_0 \beta}$	$\lceil \frac{\gamma}{\beta-\gamma} \log_{\gamma} \frac{n}{\kappa_0 \beta} \rceil_{srw}$	$c * n$	FD+XM, FB+DS [139]
BSMVB [34]	–	–	–	TMVB [57]
FLASH BUFFERING & NODE MODIFICATION				
AB-tree [64]	$\sum_{l=1}^h \frac{M^{h-l-1}}{N_l} L$	h/s_n	n	B+tree, BFTL, FD-tree
WPCB-tree [61]	h	$\lceil [1]_{sw} + 3 * n_{sp} + [n_b]_{bm} \rceil$ $\lceil [1]_{sw} + [n_b]_{bm} \rceil$	$n + B$	B-tree, the d-IPL, μ +tree
IN MEMORY BUFFERING & NODE MODIFICATION				
MB-tree [124]	$2 + \lceil \log_M \frac{2n}{M+n_l} \rceil$	$\lceil 3/n_l \rceil_w$ $+ \lceil (n_l + \lceil \log_M \frac{2n}{M+n_l} \rceil) / n_l \rceil_r$	$n + B$	BFTL, B+tree(ST), B-tree
FB-tree [75]	–	–	–	B+tree
Bw-tree [92, 93]	–	–	–	BerkeleyDB, Skip List
Bloom tree [66]	$h + p_{ip} * d + 2$	–	$n + B$	B+tree, B+tree(ST), FD-tree, MB-tree
IN MEMORY BUFFERING & IN MEMORY BATCH READ BUFFERING & NODE MODIFICATION				
PIOB-tree [125, 126]	$h - 1 + \ell_L$	$\lceil \sum_{l=1}^{h-2} \frac{1}{G(l)} + \frac{1}{G(h-1)} \rceil_r + \lceil \frac{1}{G(h-1)} \rceil_w$ $-\frac{1/M^{(\ell_L)}}{G(\log_M(\mu-B)-1)}$	$n + \mu$	BFTL, FD-tree, B+tree

- Algorithms Behind Modern Storage Systems, <https://queue.acm.org/detail.cfm?id=3220266>
- Indexing in flash storage devices: a survey on challenges, current approaches, and future trends, <https://link.springer.com/article/10.1007/s00778-019-00559-8>

Summary of Data Structures

- B+ Tree (read-optimized)
 - Fast, bounded lookup for read/get ($\log(n)$)
 - Efficient range based queries
 - But poor performance for write-heavy workloads, update bubbling (also small updates)
- Log-structured Merge (LSM) Tree (write-optimized)
 - Good performance for write-heavy workloads, large sequential log based updates
 - Ranged based queries possible
 - Read/Write amplification is a problem
- Simple hash table (hash like md5 on the key $\rightarrow$ map to a location)
 - Easy and fast lookup and retrieval ($O(1)$)
 - Limited range based query support (need additional indexing)
 - Tradeoff between (memory usage, and flash I/O)

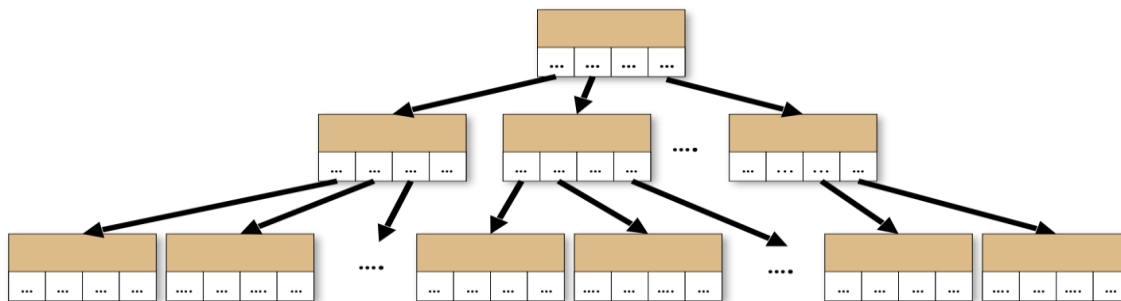
An example

- Index all Integers from 100 to 1M

Data =

100	101	102	103	104	105	...	1M
-----	-----	-----	-----	-----	-----	-----	----

B-Tree?



An example

- Index all Integers from 100 to 1M

Data =

100	101	102	103	104	105	...	1M
-----	-----	-----	-----	-----	-----	-----	----

Data[key - 100]

An example

- What about even integers?

Data =

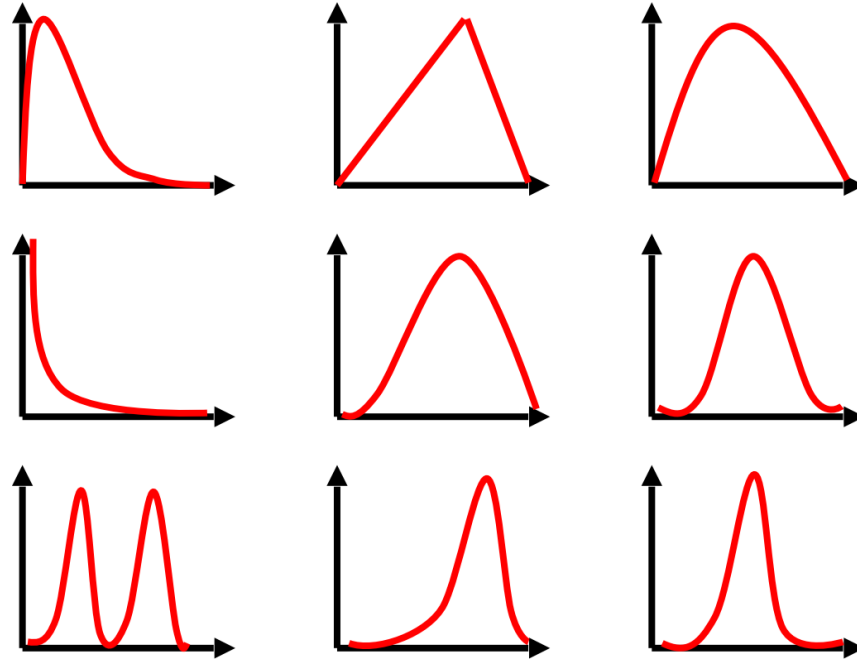
100	101	102	103	104	105	...	1M
-----	-----	-----	-----	-----	-----	-----	----

Data =

100	102	104	106	108	110	...	1M
-----	-----	-----	-----	-----	-----	-----	----

$\text{Data}[(\text{key} - 100) / 2]$

Other distributions?



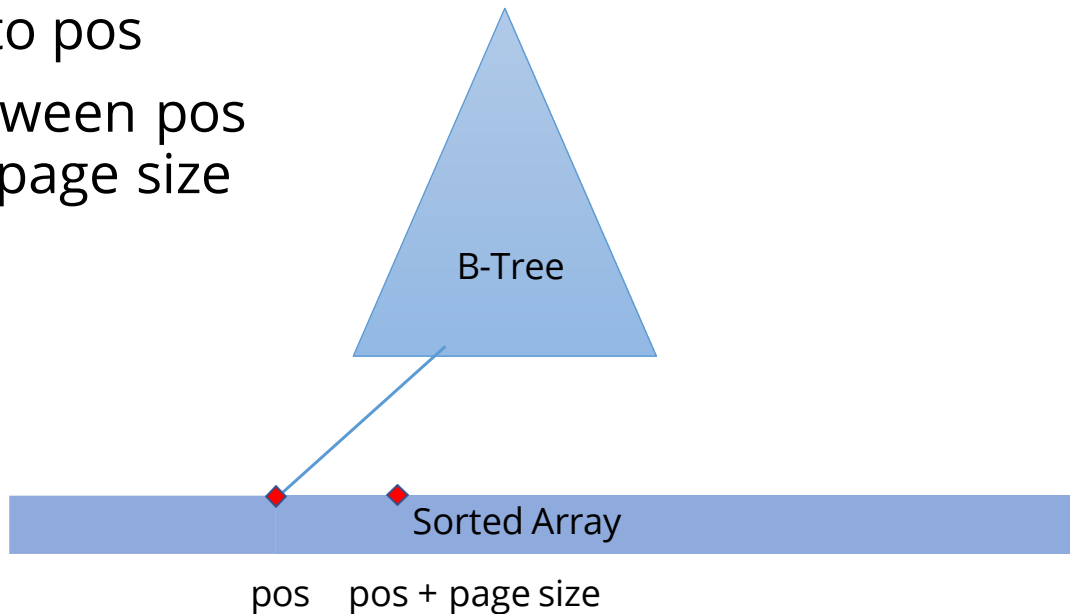
Key Insight

- Traditional data structures make no assumption about data
- Optimized for worst case

- Building a system from scratch for every use case is not scalable
- What if we could learn the data distribution?

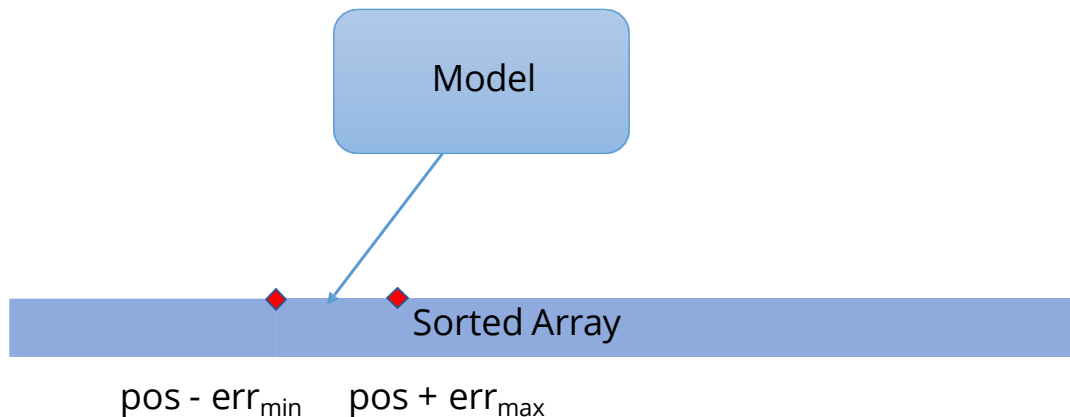
B-Tree as a model

- Maps key to pos
- Search between pos and pos + page size



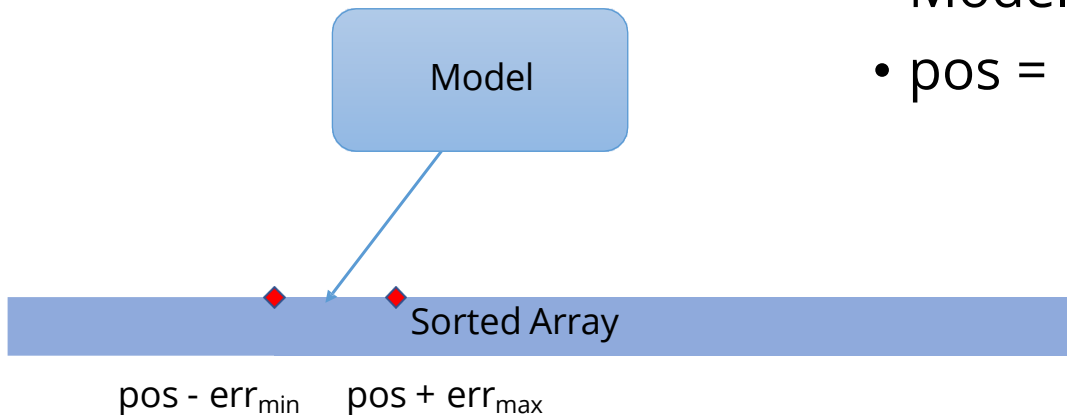
Index as a model

- Model maps key to pos
- Search in $[\text{pos} - \text{err}_{\min}, \text{pos} + \text{err}_{\max}]$
- $\text{err}_{\min}$ and $\text{err}_{\max}$ are known from training

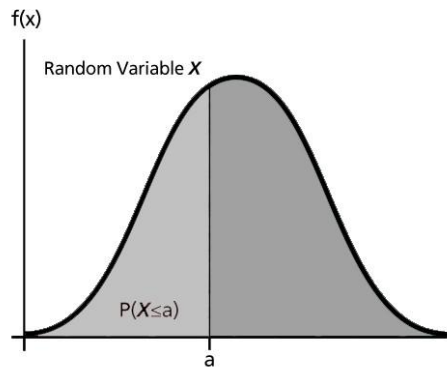


What is the model estimating?

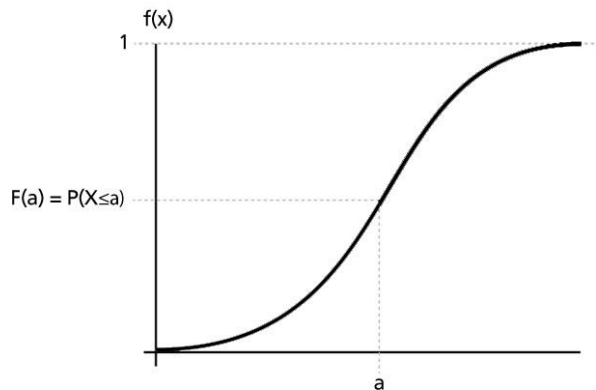
- Predicting the position of a key
- Modeling the CDF of the keys
- $\text{pos} = P(X \leq \text{key}) * \text{\#keys}$



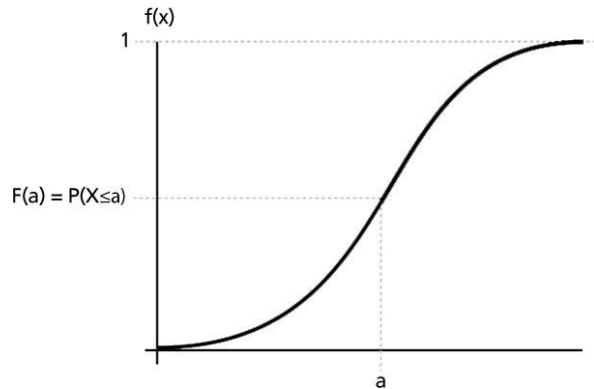
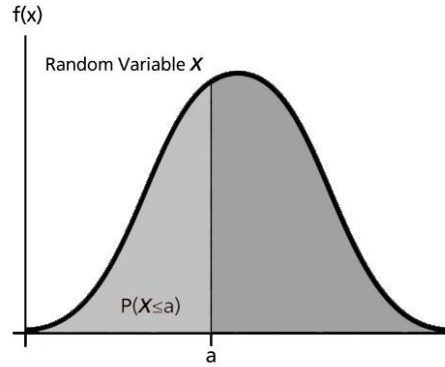
What is the model estimating?



- Predicting the position of a key
- Modeling the CDF of the keys
- $\text{pos} = P(X \leq \text{key}) * \text{\#keys}$
 $= F(\text{key}) * \text{\#keys}$



LookUp(key)



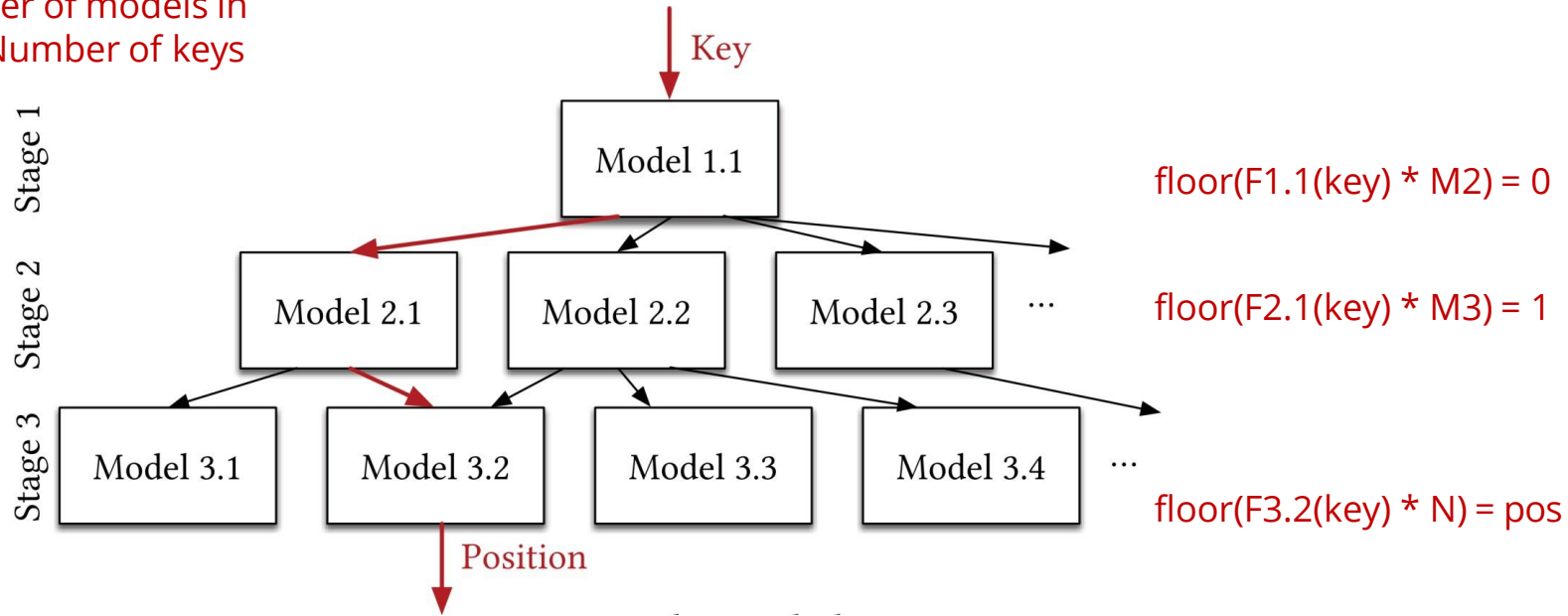
- Use the CDF model to predict the position of the key
- $\text{pos} = F(\text{key}) * \text{\#keys}$
- Scan from $[\text{pos} - \text{errmin}, \text{pos} + \text{errmax}]$

Recursive Model Index (RMI)

$F_{l,k}$ - CDF of $M_{l,k}$

M_l - Number of models in

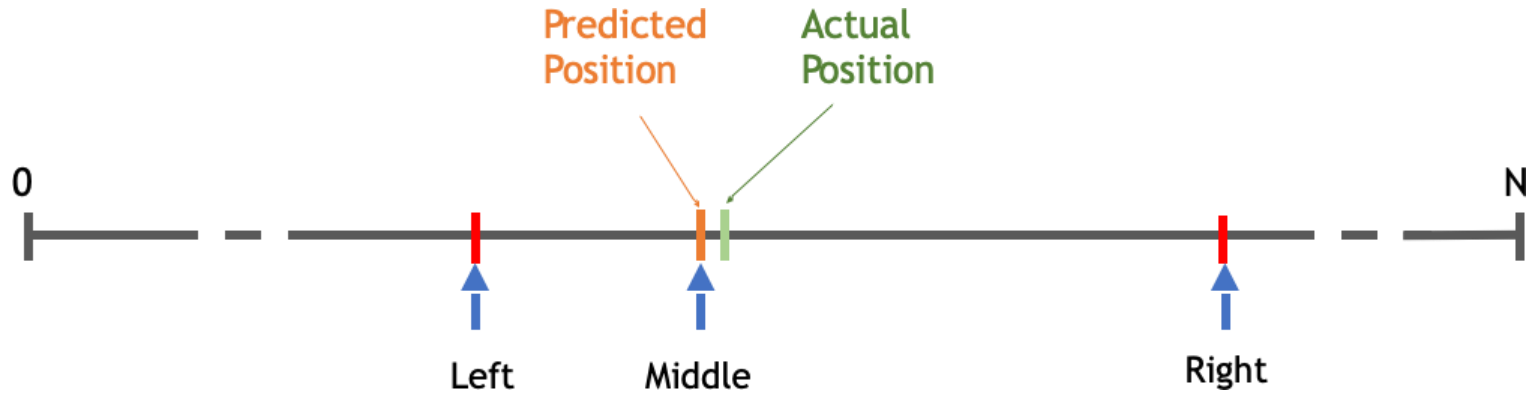
level l N - Number of keys



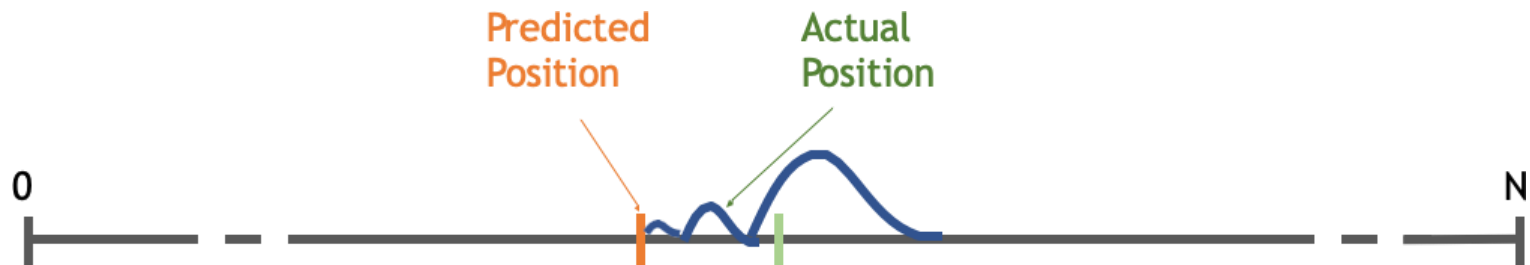
Key Idea

- When CPU cycles are cheap relative to memory accesses, compute- intensive function approximations can be beneficial for lookups
- ML models may be better at approximating some functions than existing data structures
- Power of continuous functions

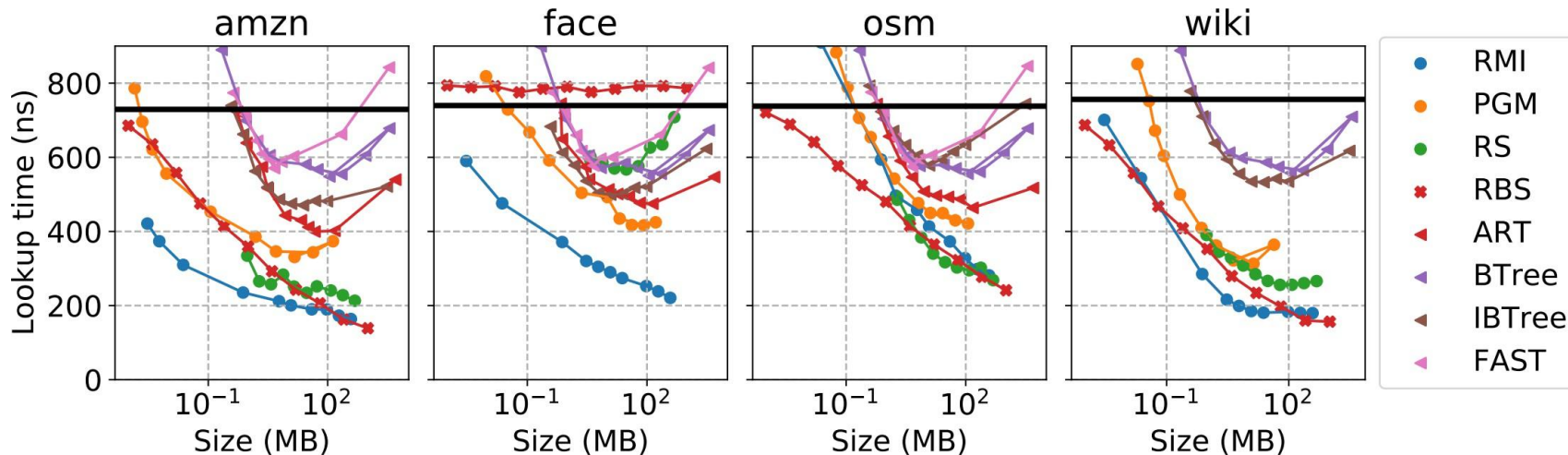
Binary Search



Exponential Search



Results



Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. 2020. Benchmarking learned indexes. *Proc. VLDB Endow.* 14, 1 (September 2020), 1–13. DOI:<https://doi.org/10.14778/3421424.3421425>

What you should know from this lecture

1. The idea of B+ Tree, LSM Tree, Hash Tables and learned index
2. Choices these data structures (B+ Tree, LSM, Hash Table, learned index)
3. What advantages and disadvantages they offer when implementing them over NAND flash
4. Key problem and solution: SILK
5. Key problem and solution: WiscKey
6. What is read/write amplification in LSM tree (or in any data structure)
7. The RUM Conjecture